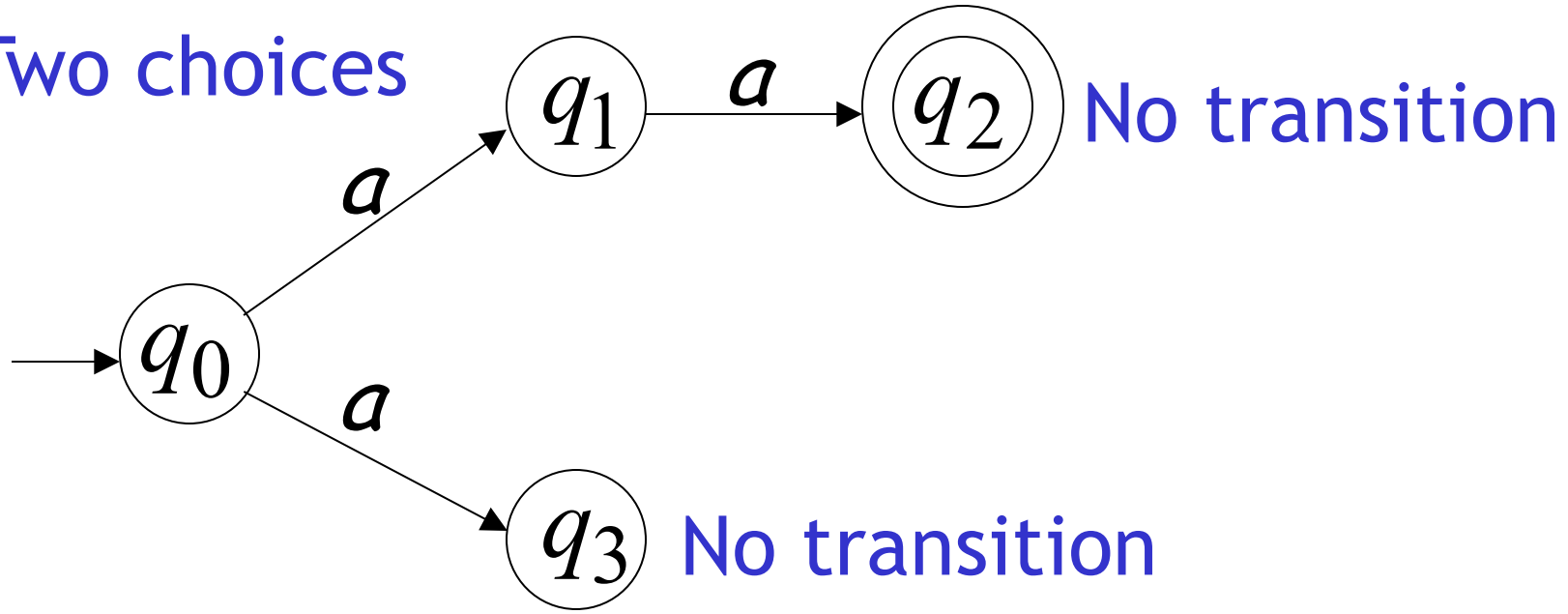
Non-Deterministic Finite Automata

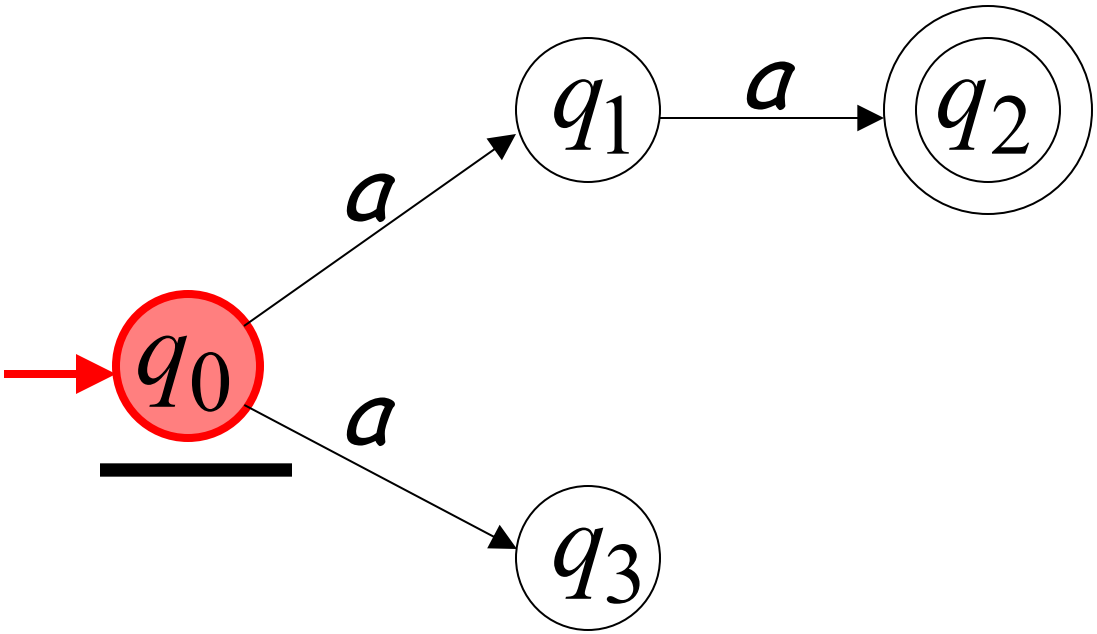
Nondeterministic Finite Automaton (NFA)

Alphabet = $\{a\}$

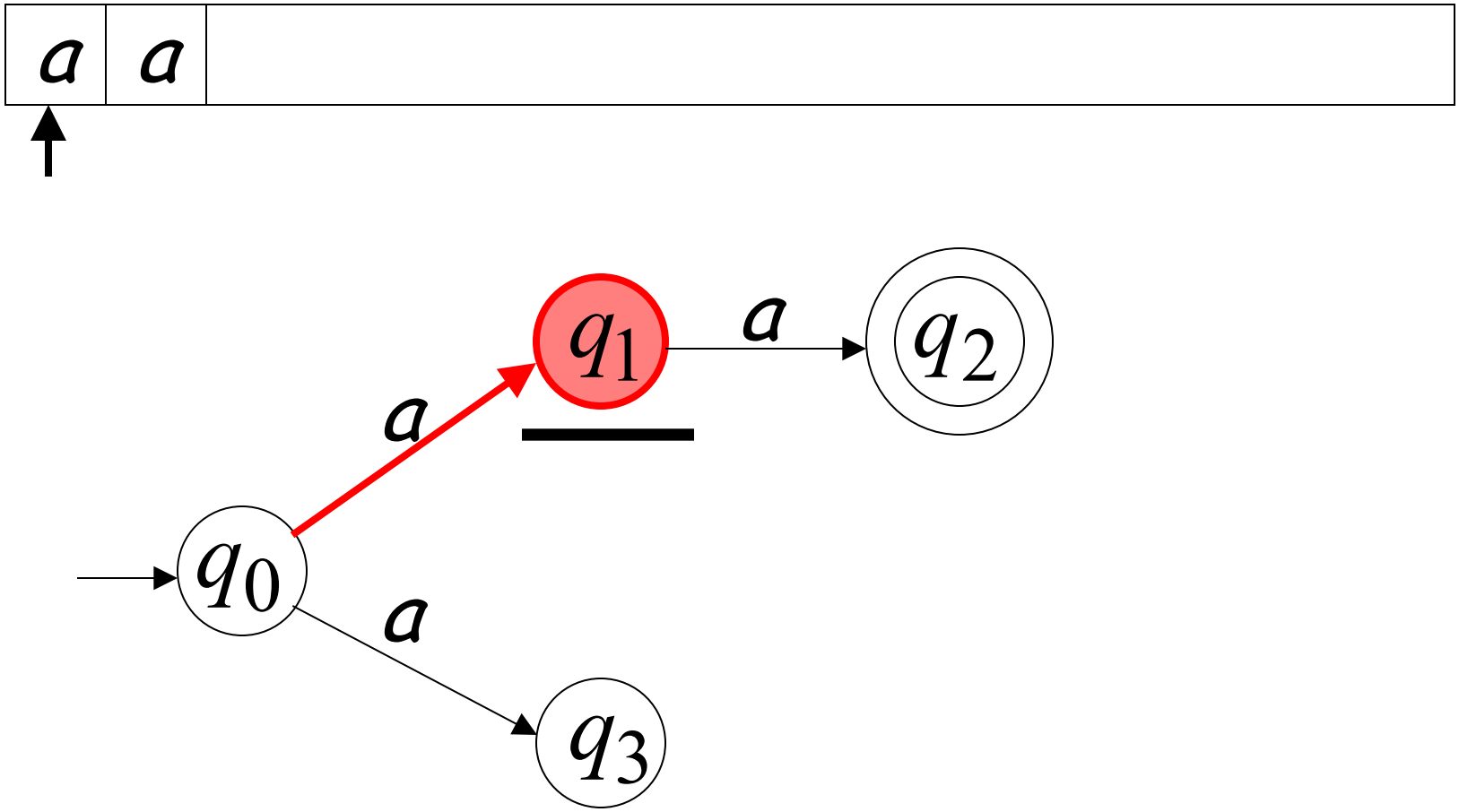


First Choice

a	a	



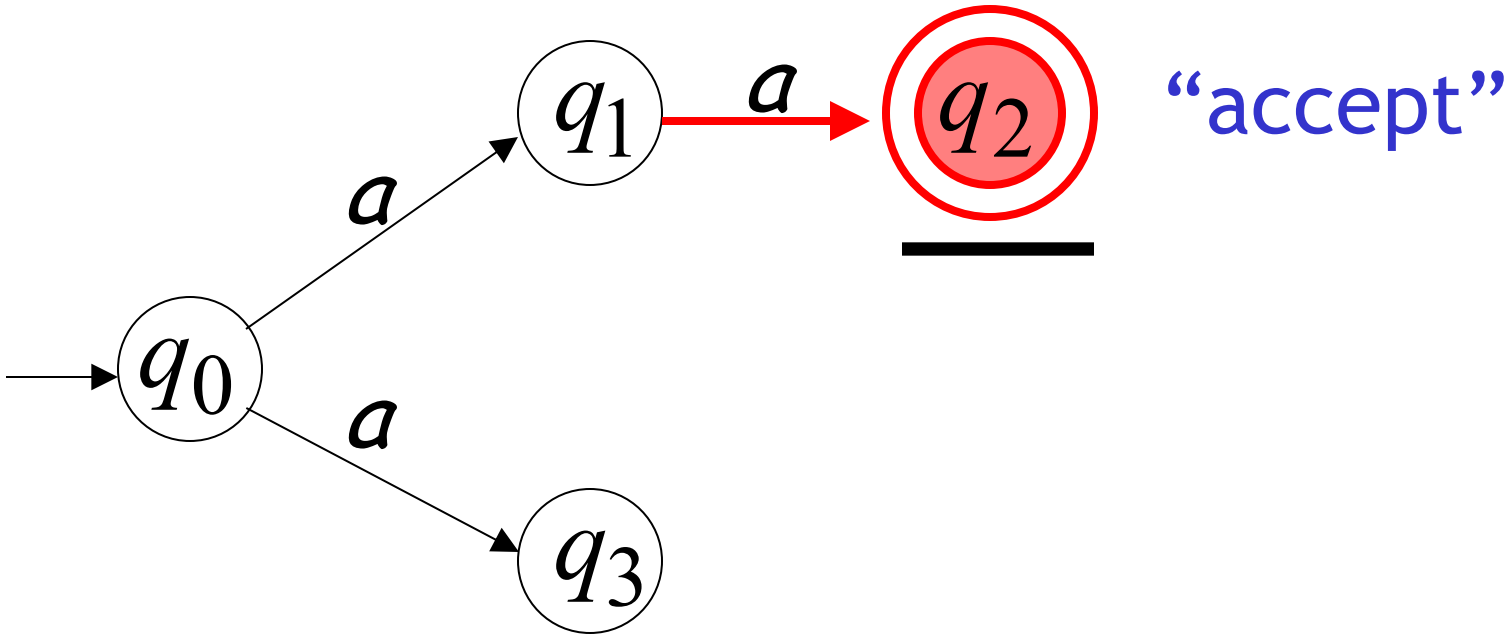
First Choice



First Choice

All the symbols in the input string is consumed.

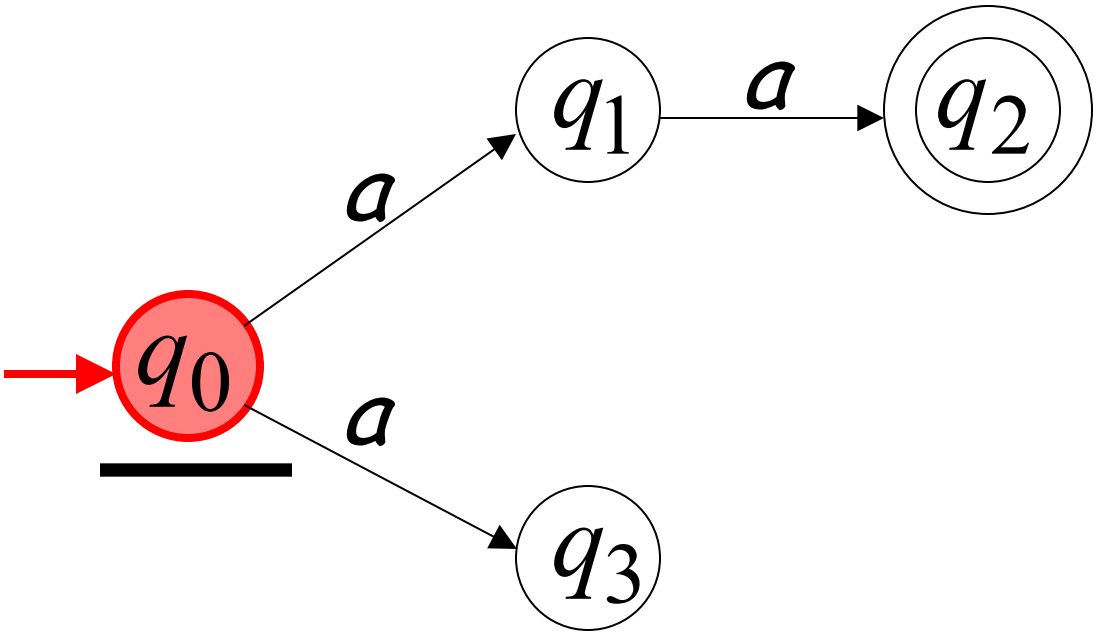




"accept"

Second Choice

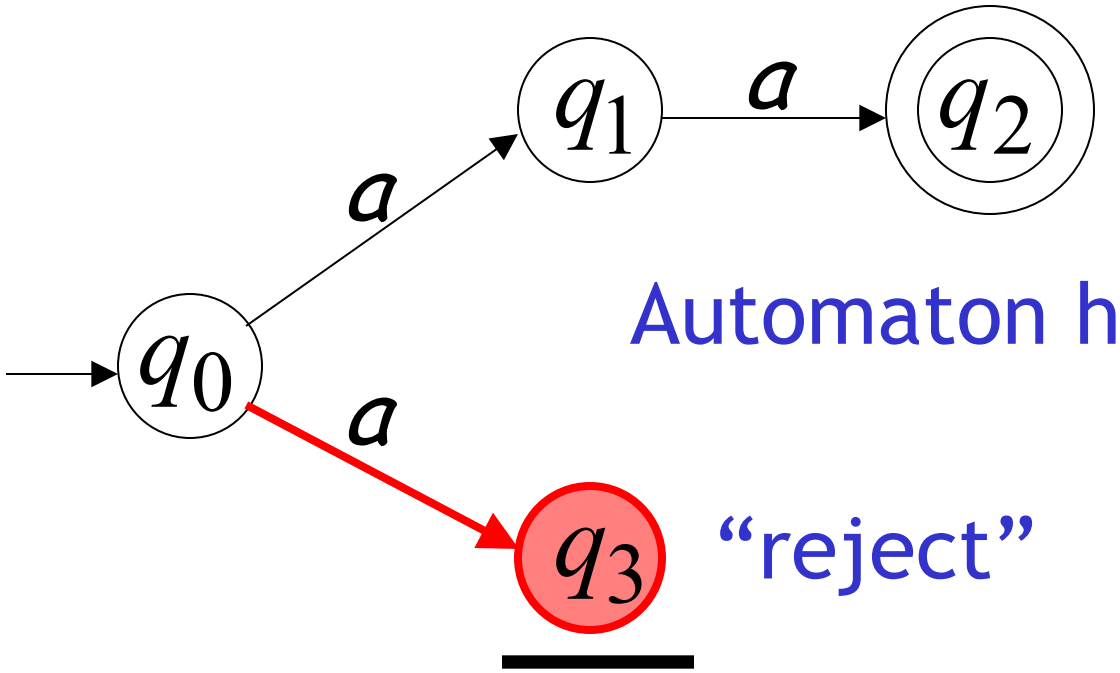
a	a	



Second Choice

Input string cannot be consumed.





Automaton halts.

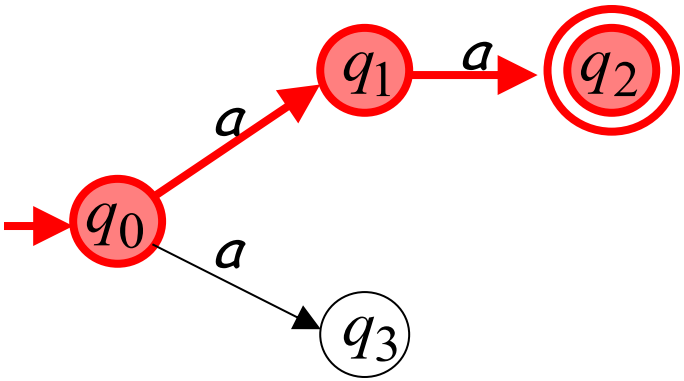
"reject"

A NFA accepts a string:

If there is at least one computation path of the NFA that accepts the input string:

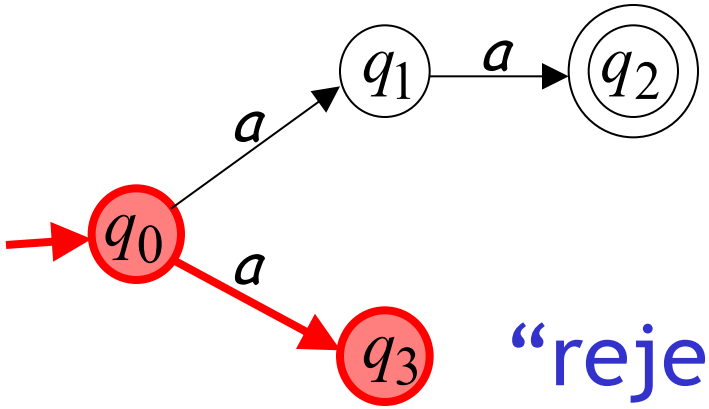
All the symbols of the input string are processed and the last state is an accept state.

aa is accepted by this NFA:



“accept”

because this computation accepts aa ,

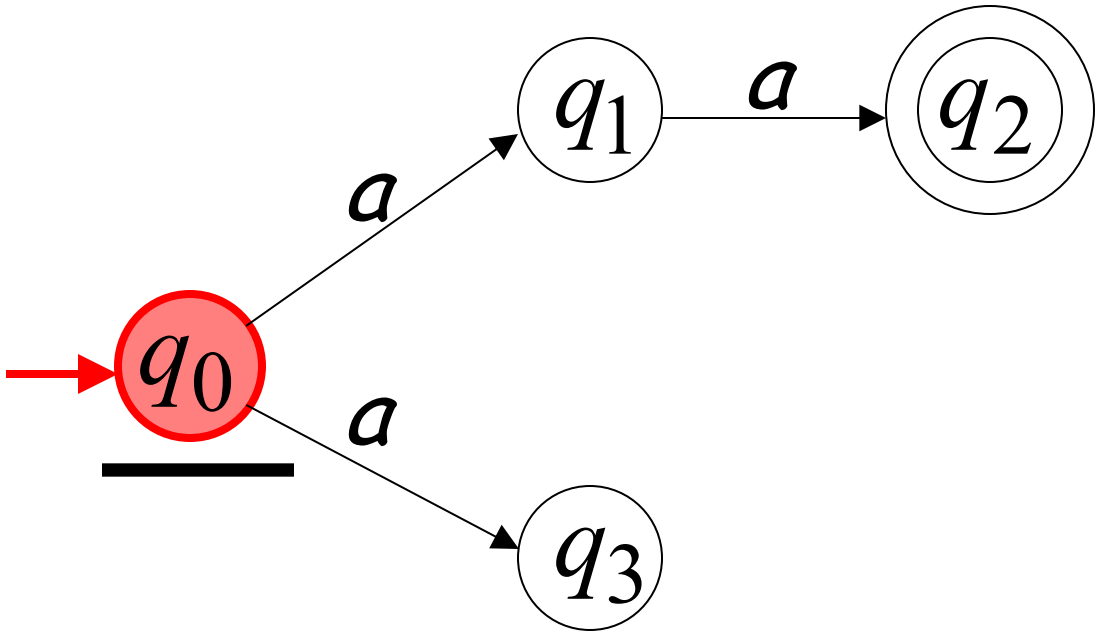


“reject”

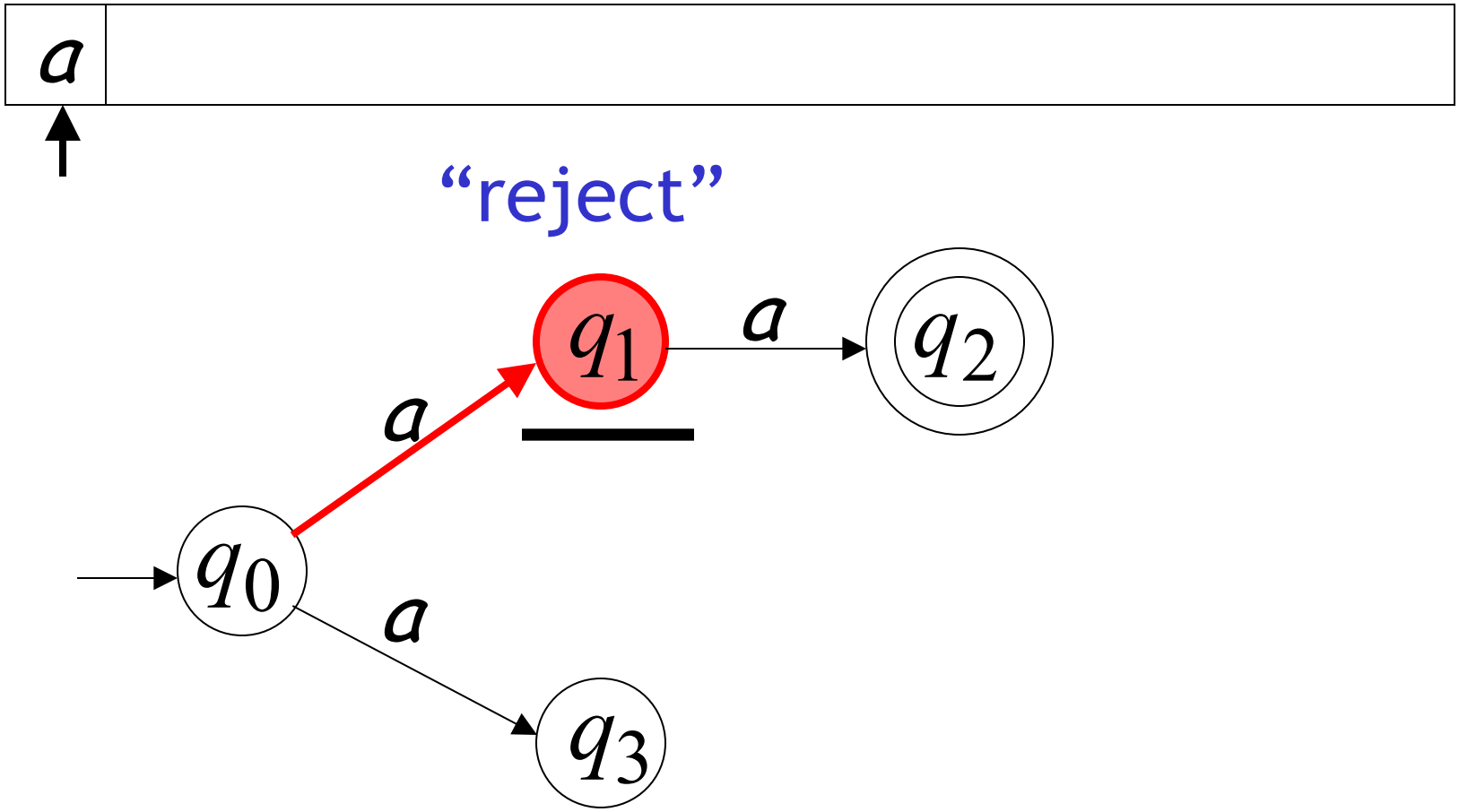
this computation is ignored.

Rejection example

a	

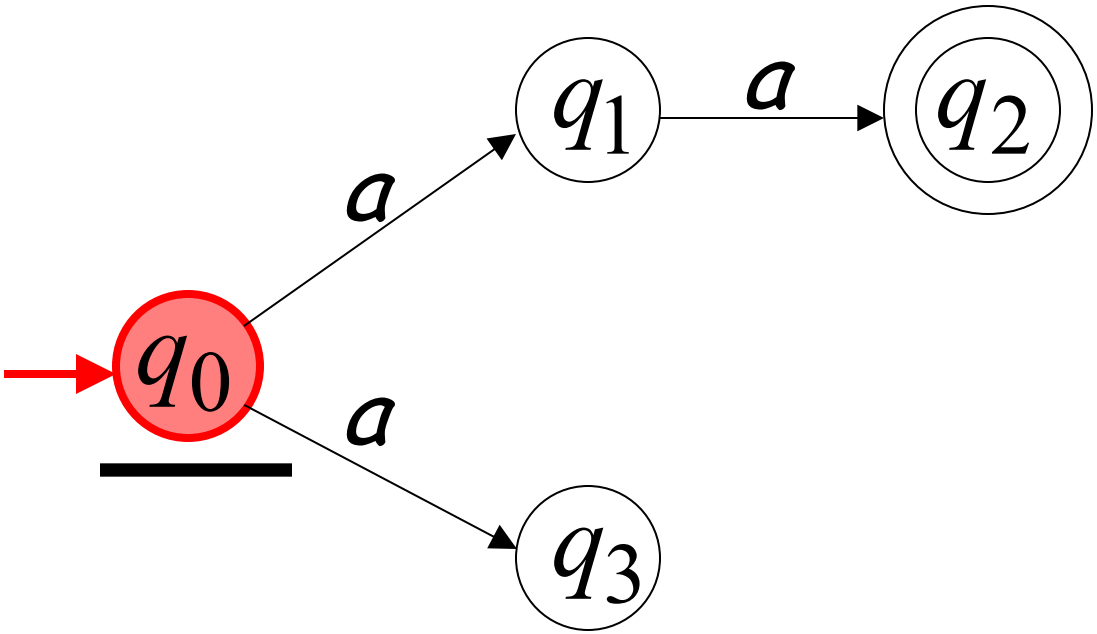


First Choice

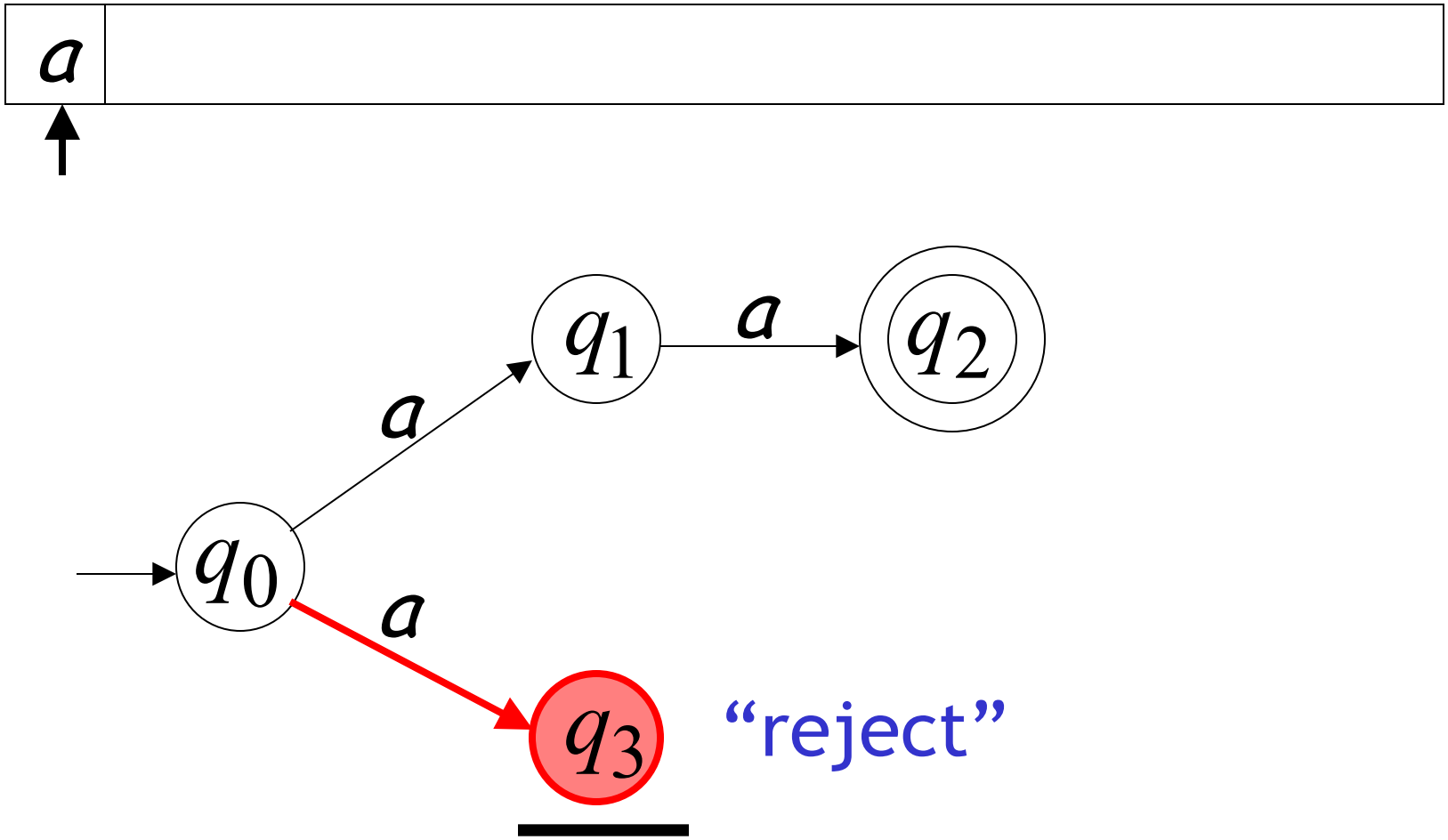


Second Choice

a	

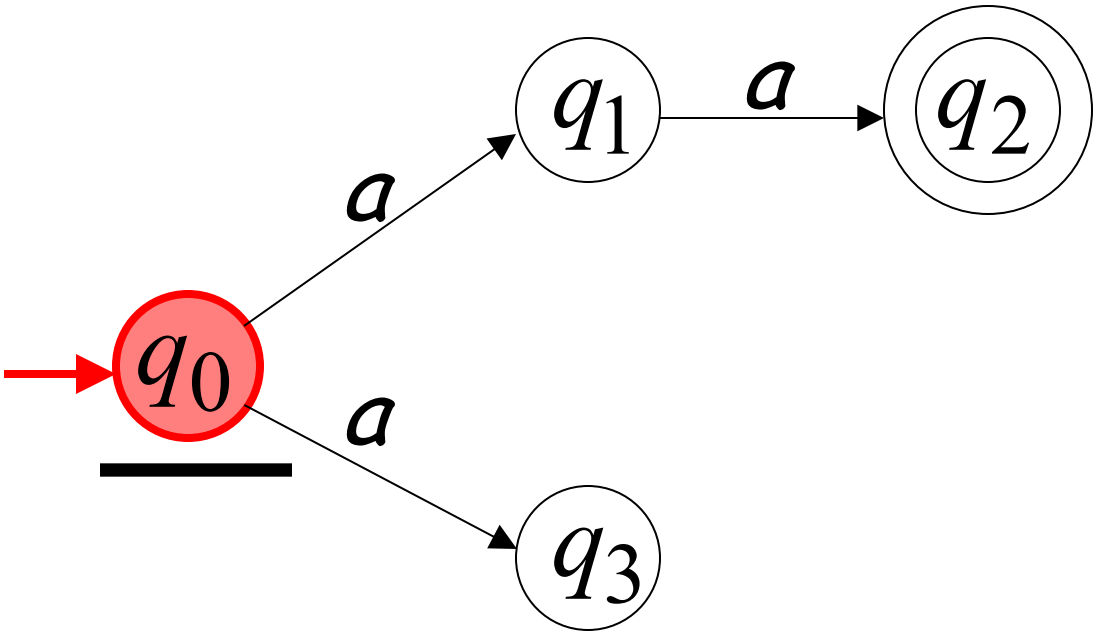


Second Choice

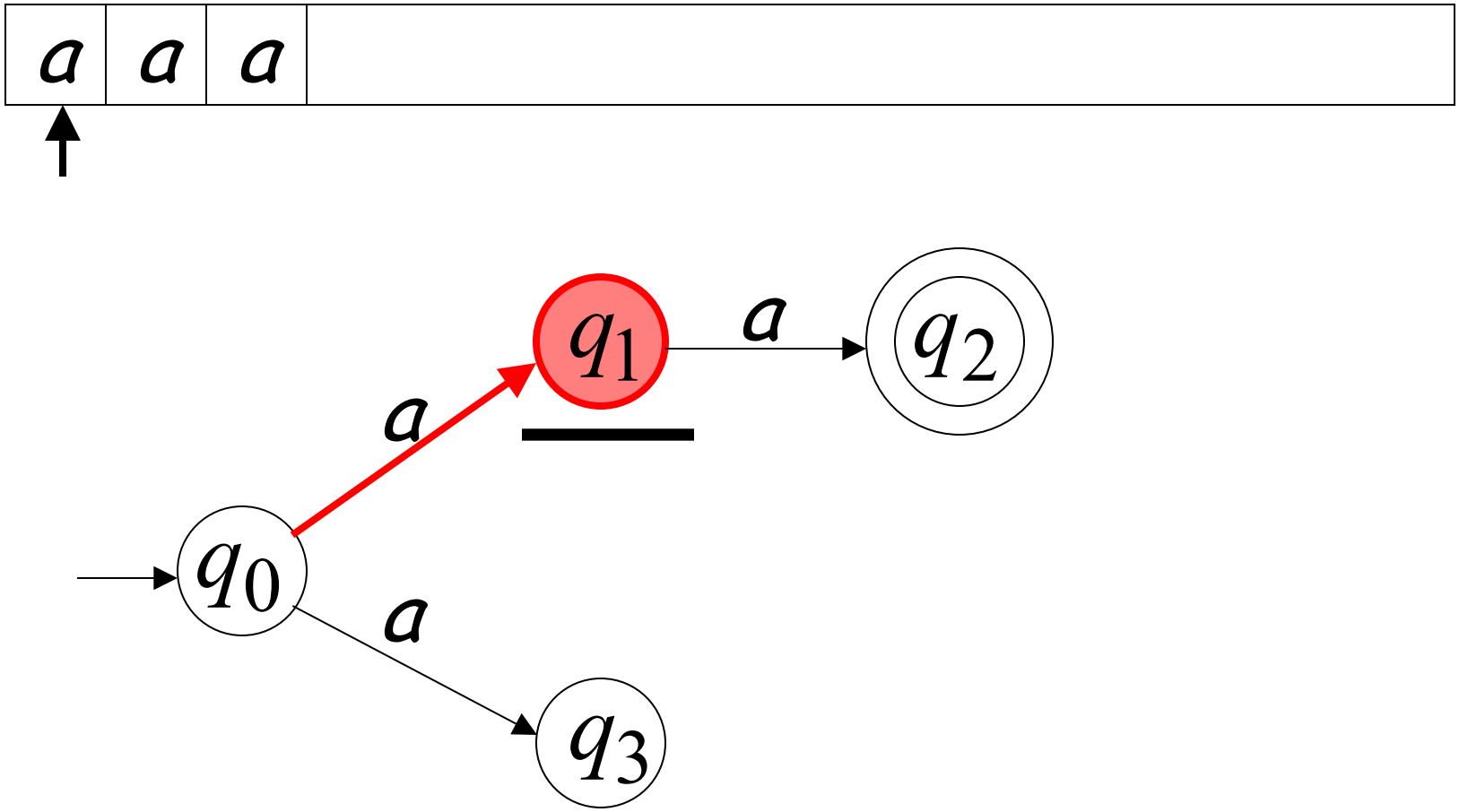


Another Rejection Example

a	a	a	



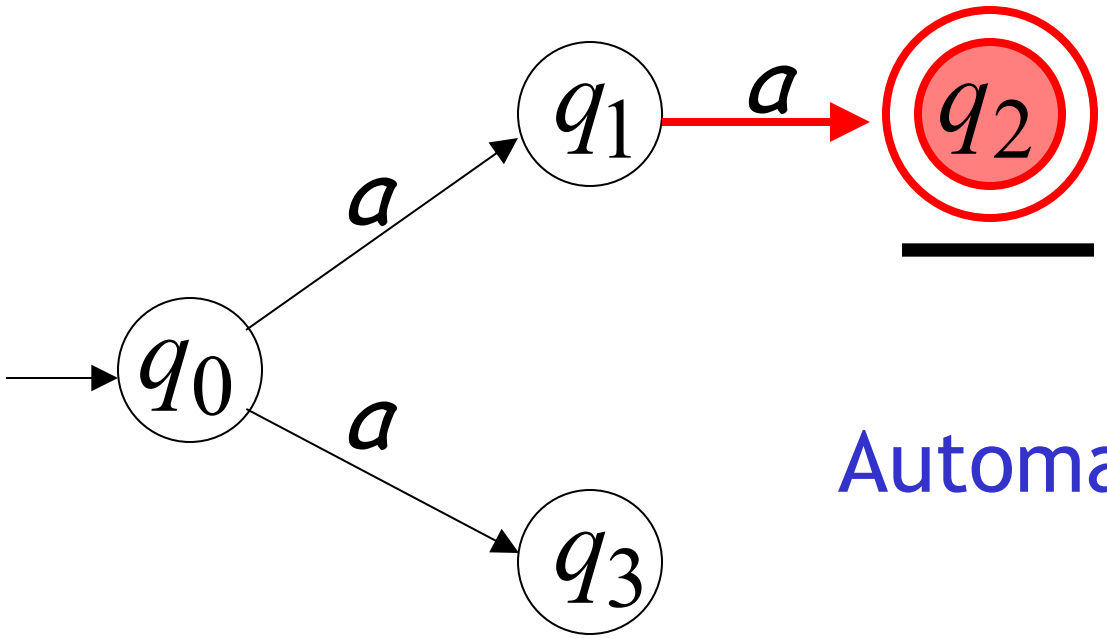
First Choice



First Choice

Input string cannot be consumed.



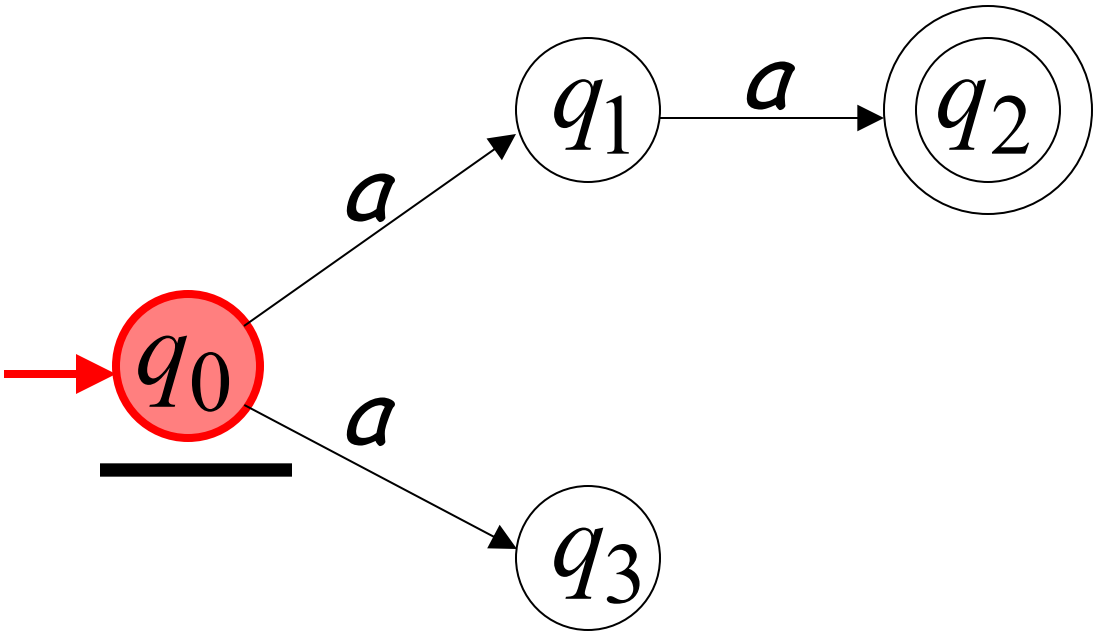


"reject"

Automaton halts.

Second Choice

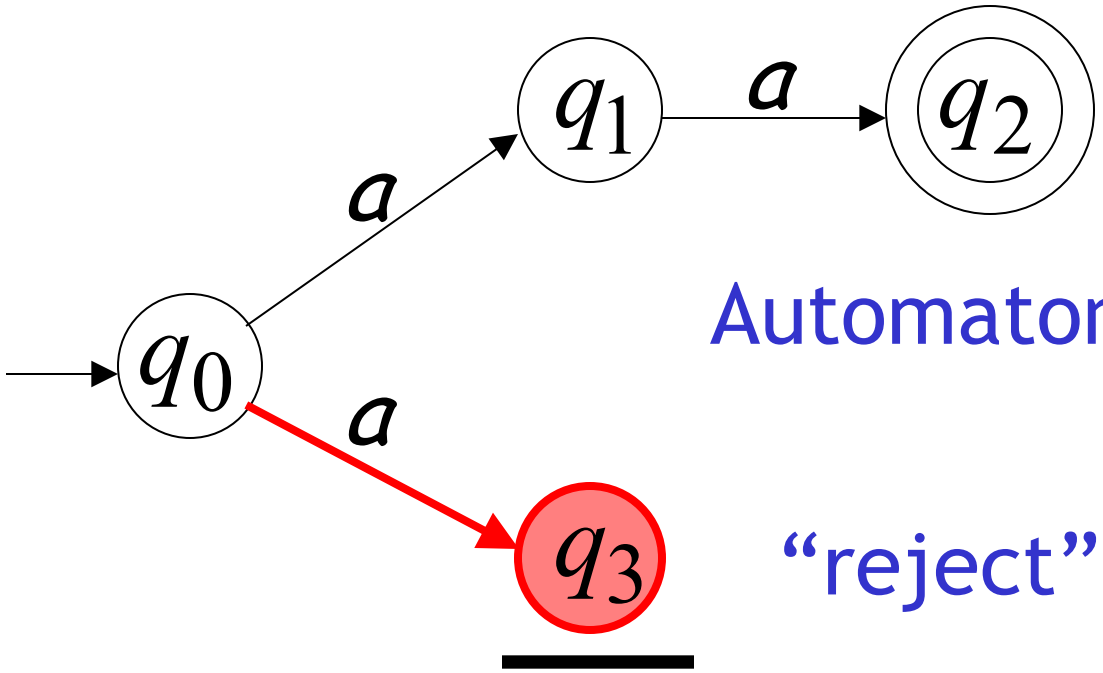
a	a	a	



Second Choice

Input string cannot be consumed.





Automaton halts.

"reject"

A NFA rejects a string:

if there is no computation of the NFA that accepts the string.

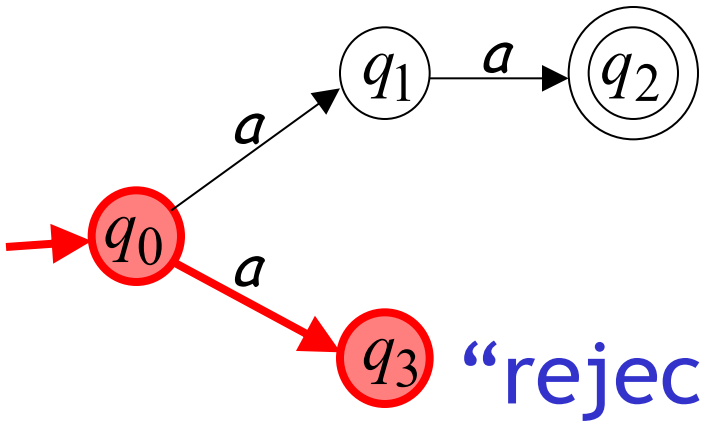
For each possible computation path:

- All the symbols of the input string is consumed and the automaton is in a non-accept state.

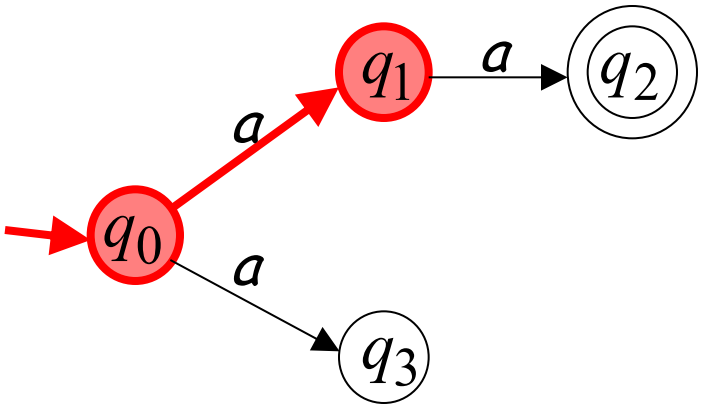
OR

- The input string cannot be consumed.

a is rejected by this NFA:

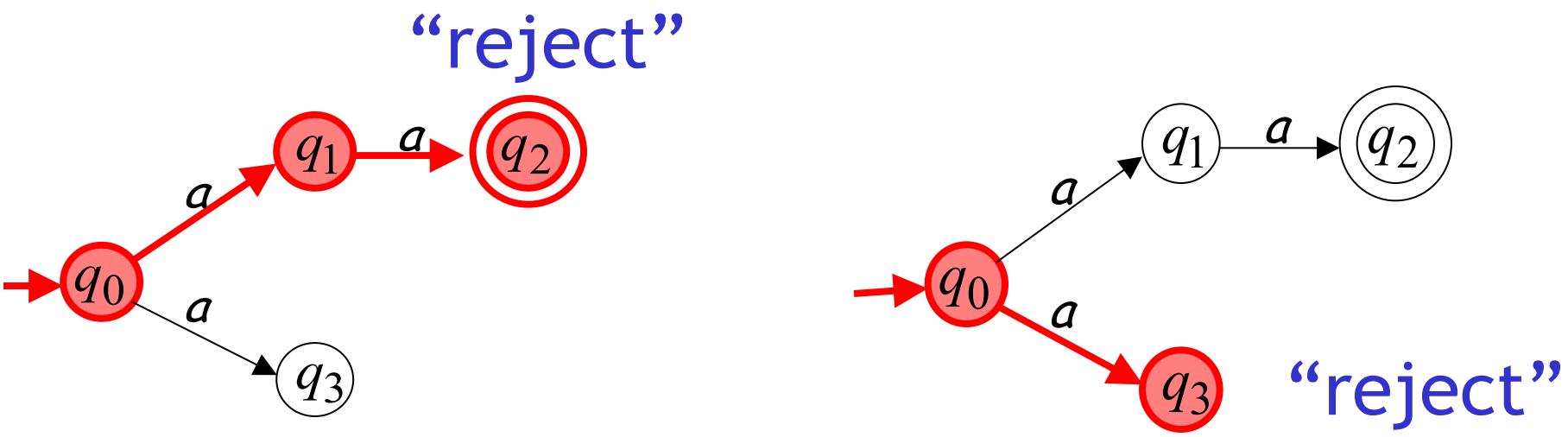


“reject”



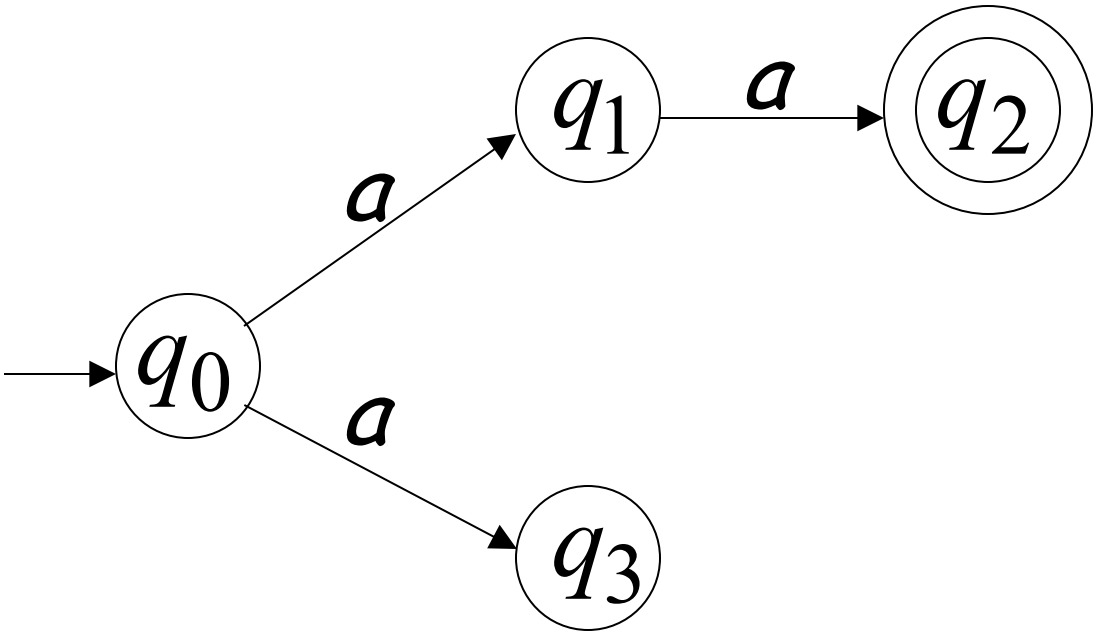
All possible computations lead to rejection.

aaa is rejected by this NFA:

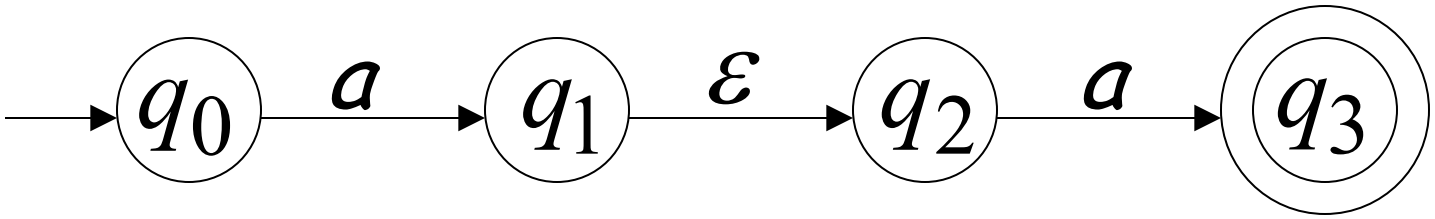


All possible computations lead to rejection.

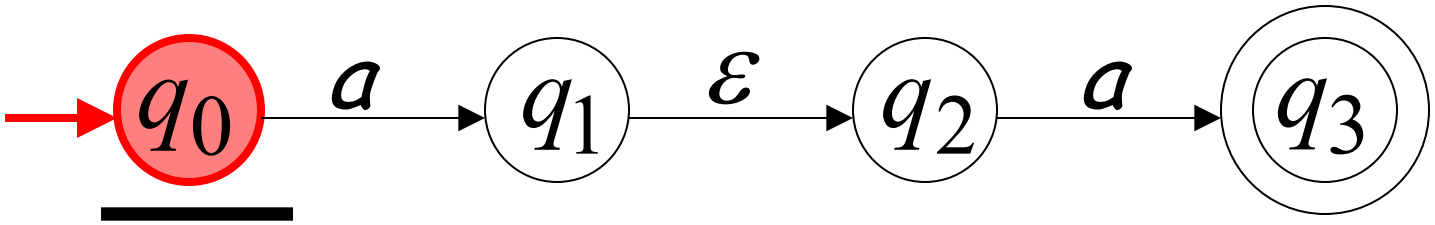
Language recognized by this NFA: $L = \{aa\}$

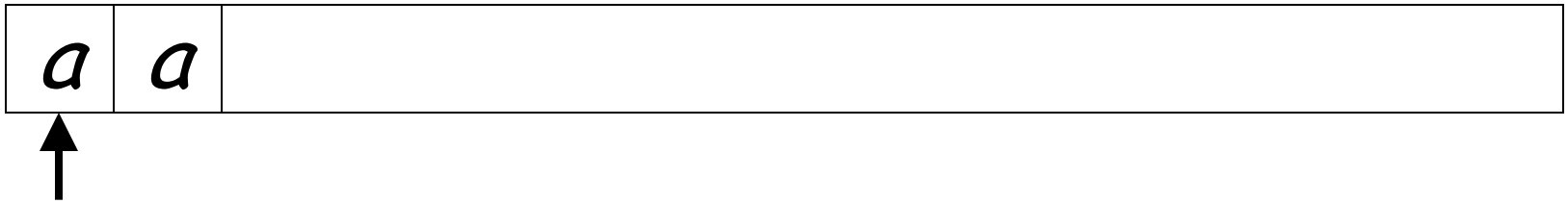


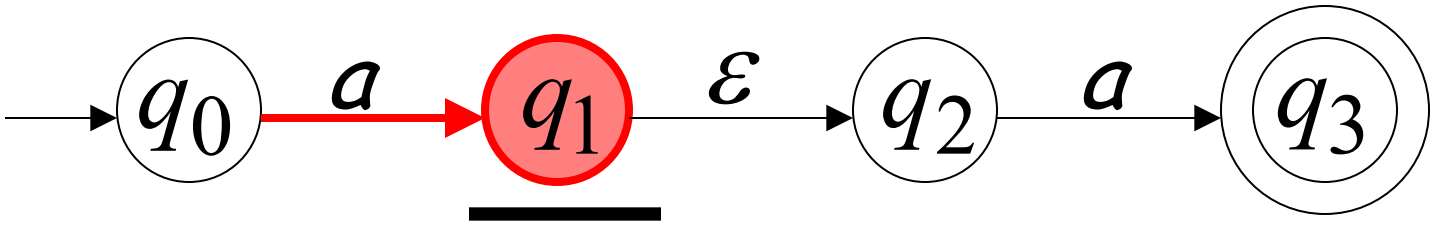
Epsilon (Null) Transitions



a	a	

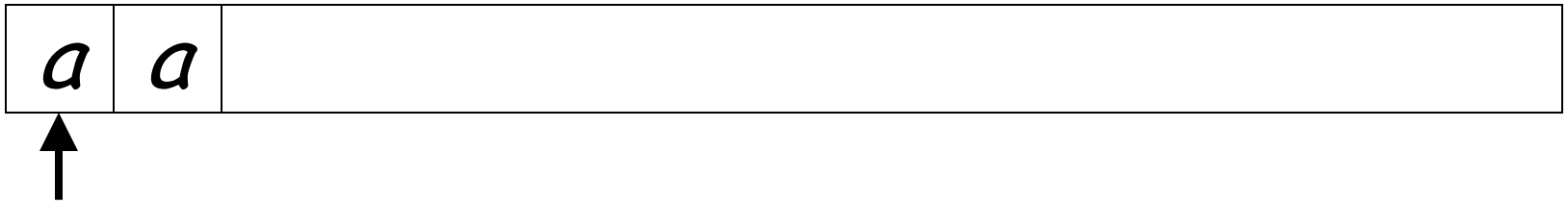


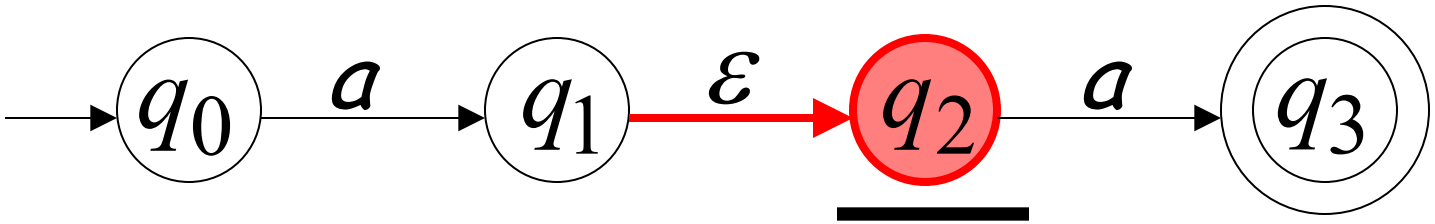

a
a


q_0
a
q_1
ε
q_2
a
q_3

Epsilon (Null) Transition

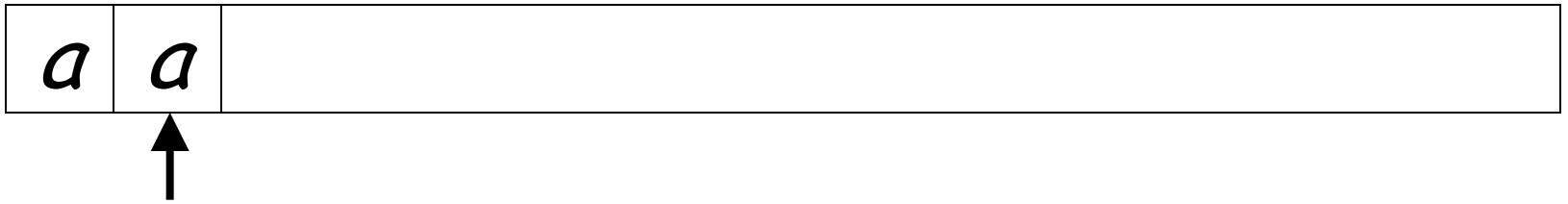
Input tape read head does not move.



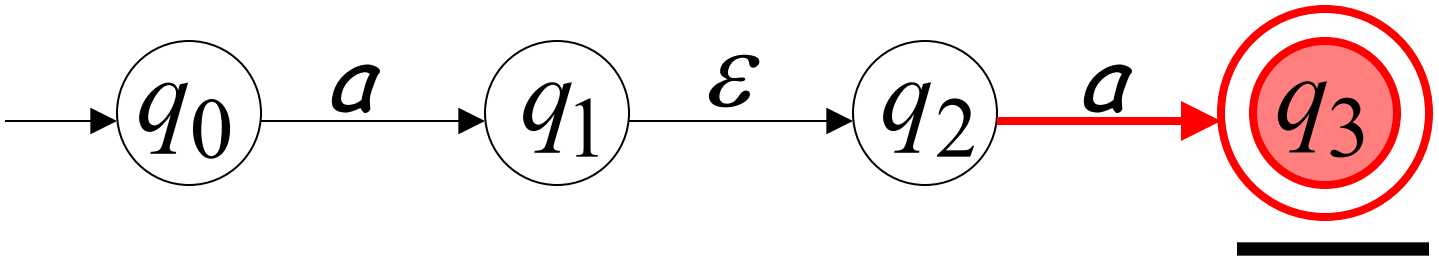


Automaton changes state.

All the symbols in the input string is consumed.



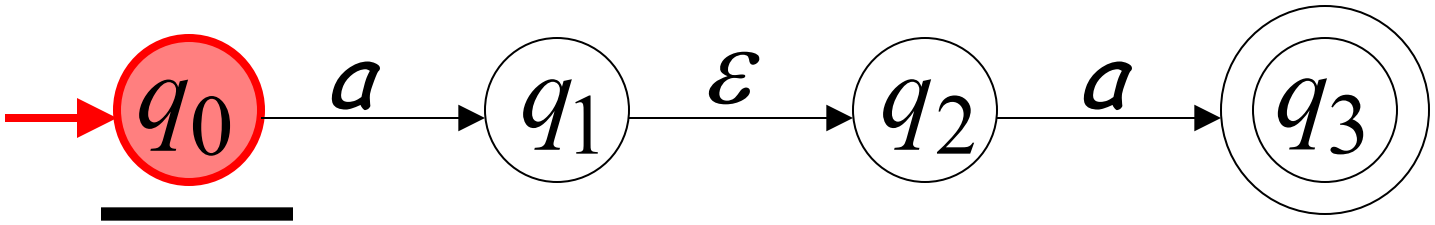
"accept"

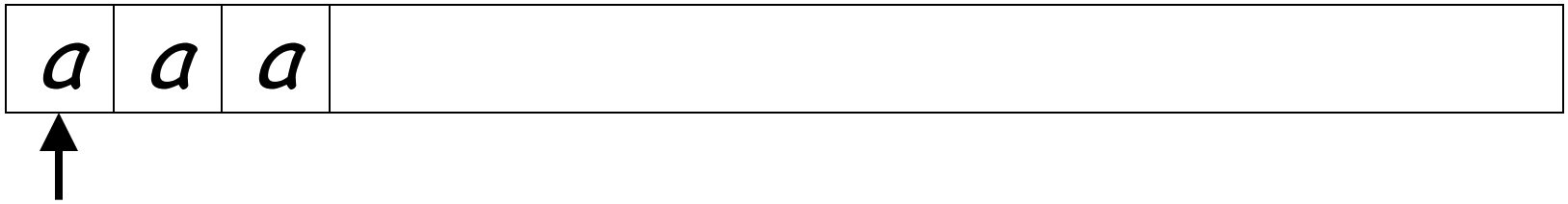


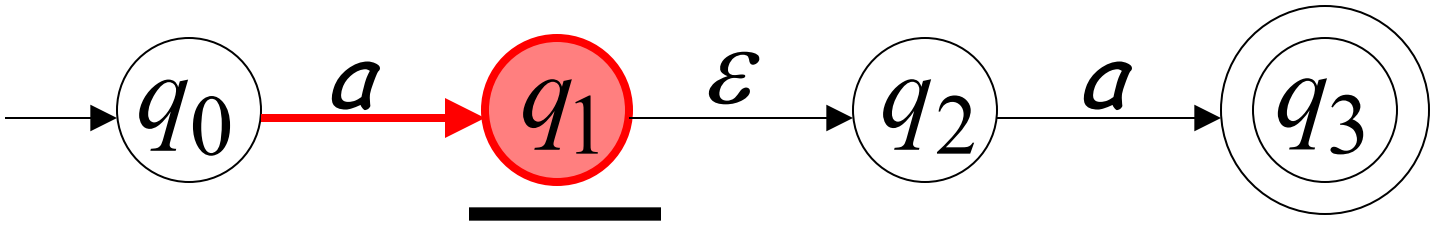
String aa is accepted.

Rejection Example

a	a	a	

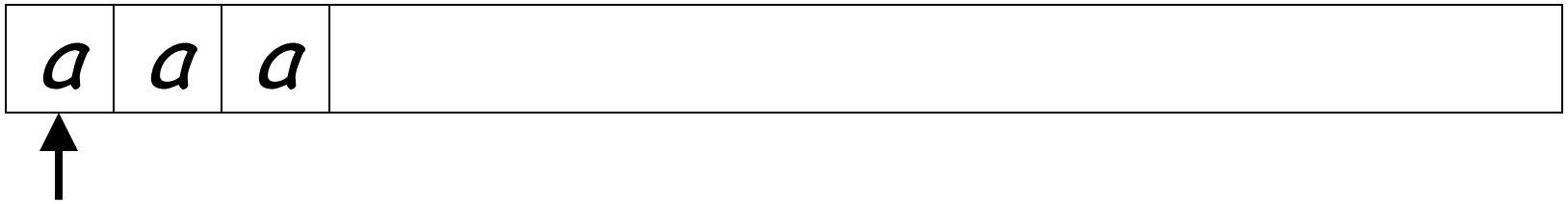


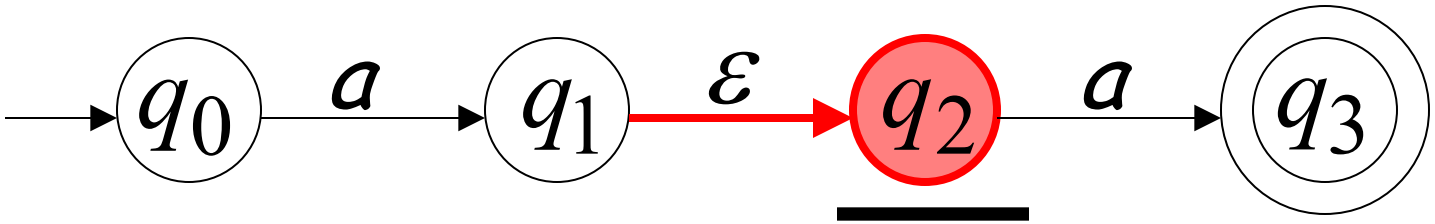

a
a
a


q_0
a
q_1
ε
q_2
a
q_3

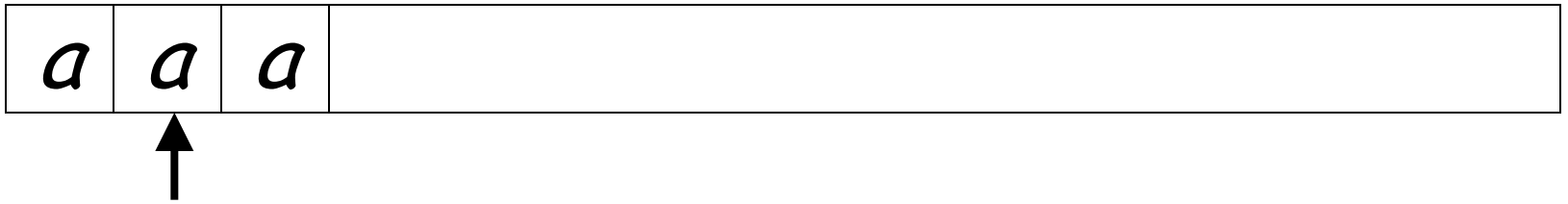
Epsilon (Null) Transition

Input tape read head does not move.



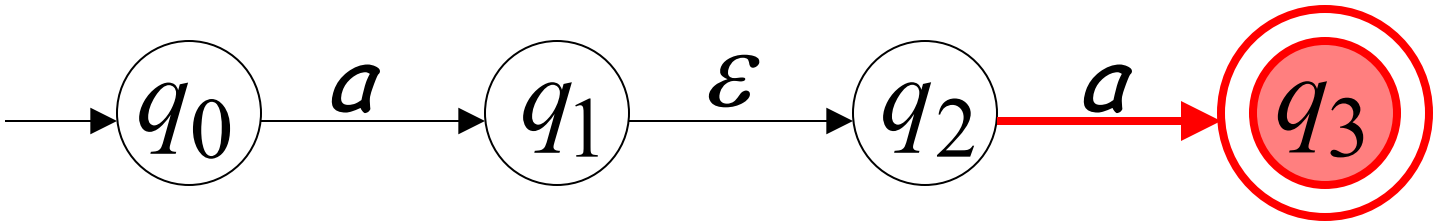


Input cannot be consumed



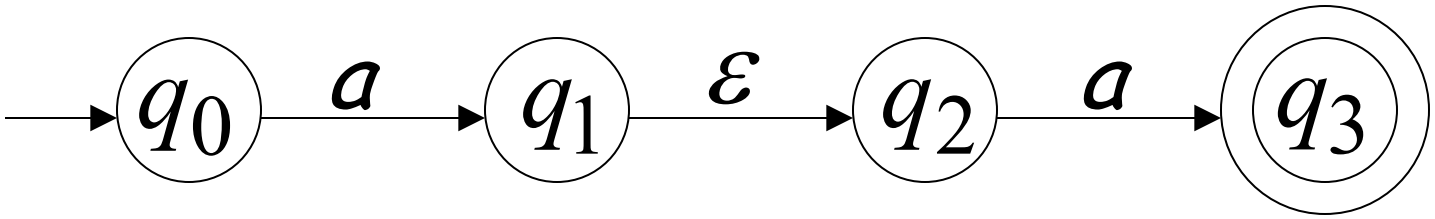
Automaton halts

"reject"

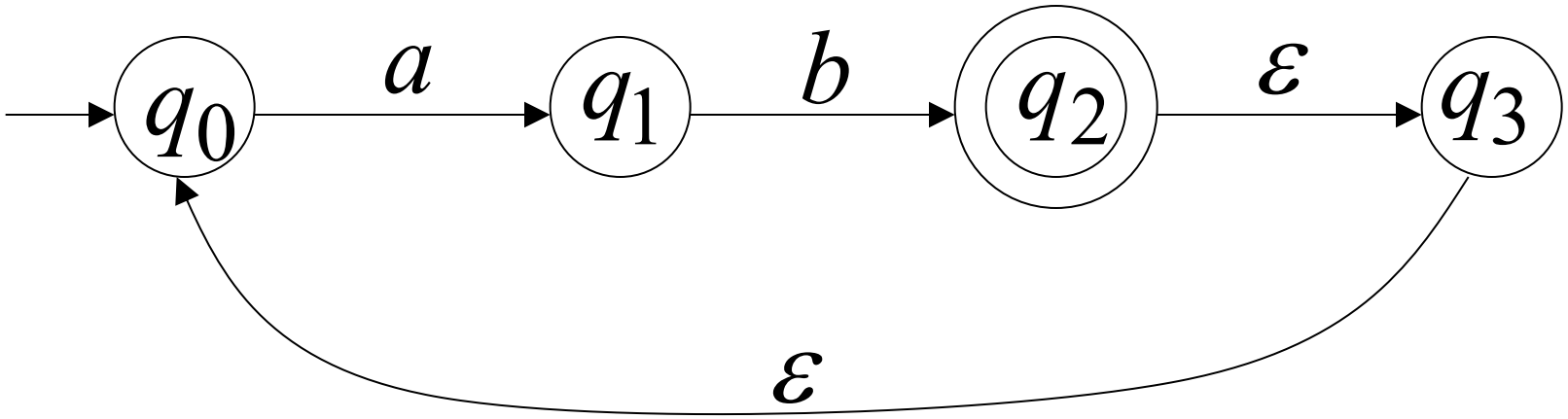


String aaa is rejected

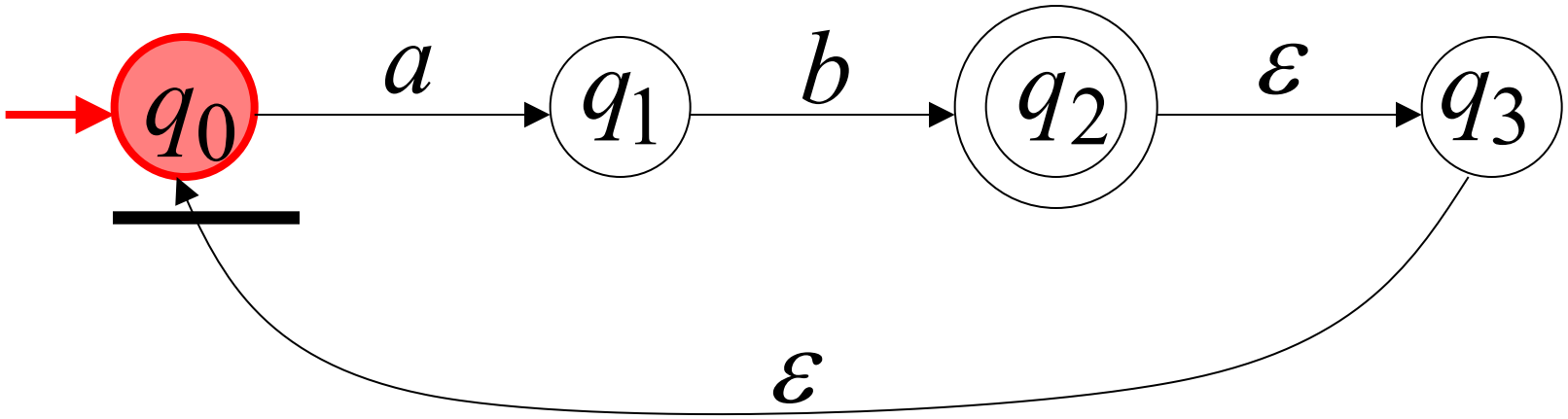
Language recognized by this NFA: $L = \{aa\}$

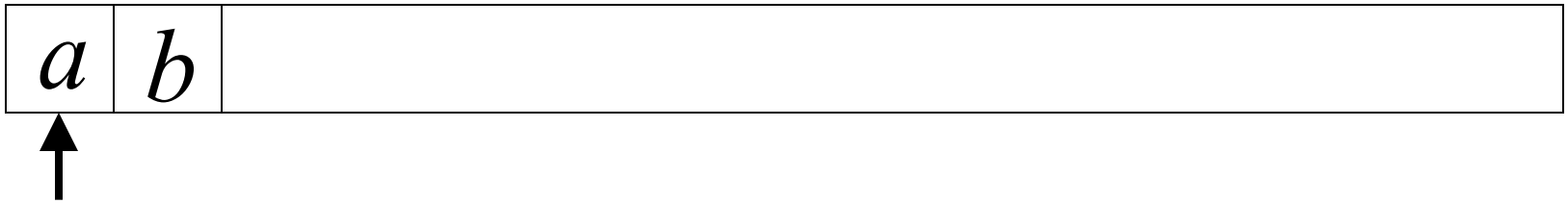


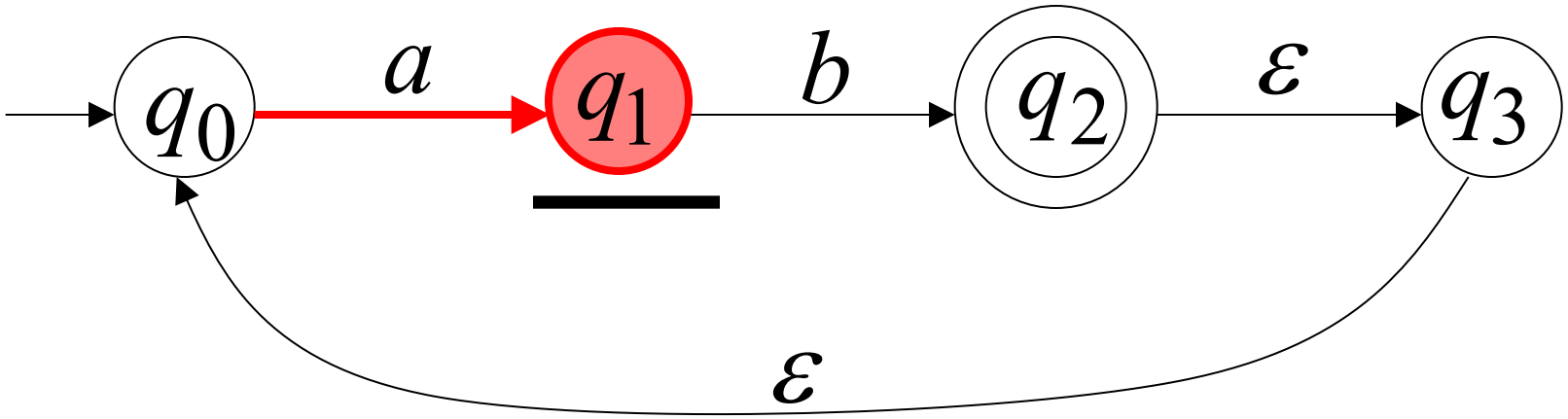
Another NFA Example

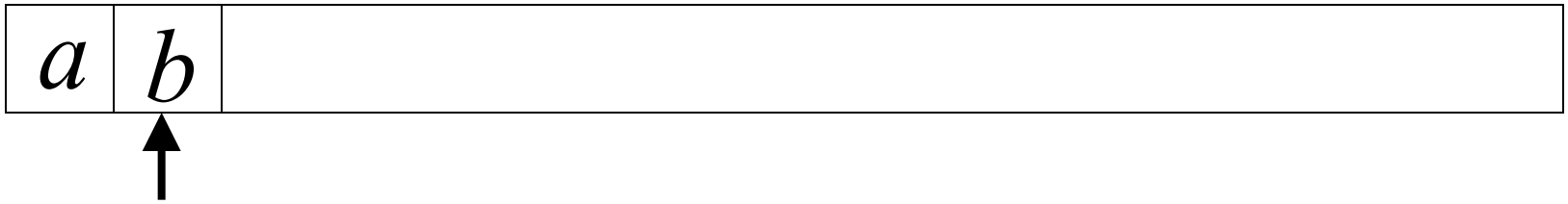


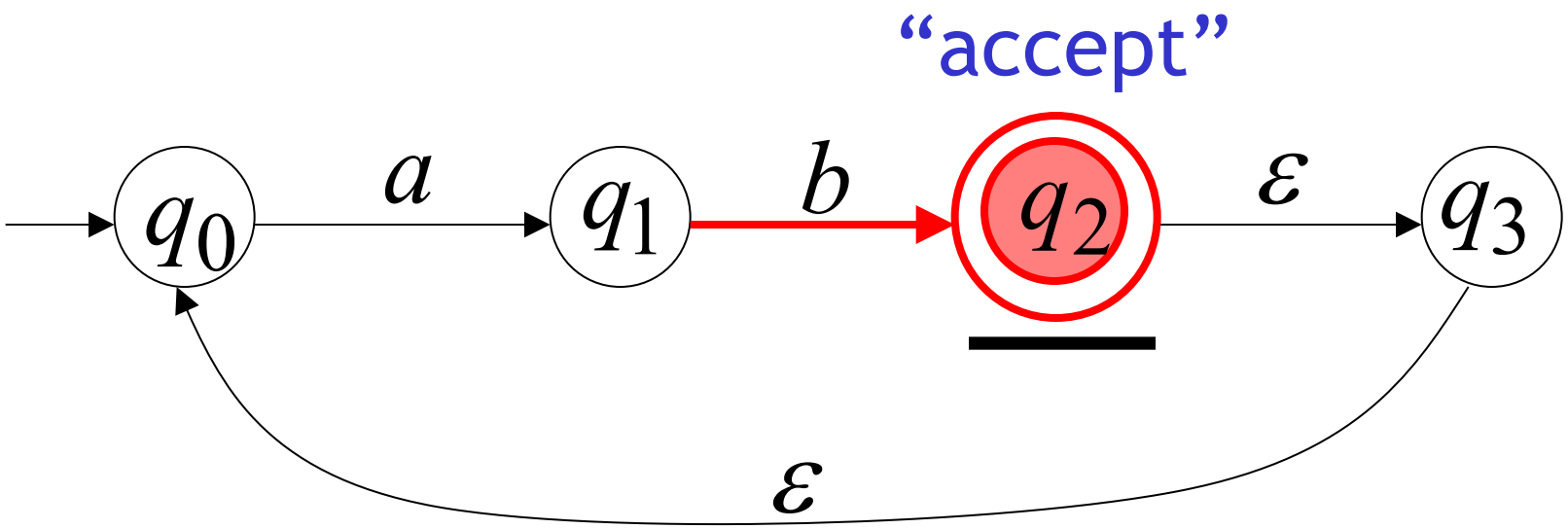
a	b	




a
b

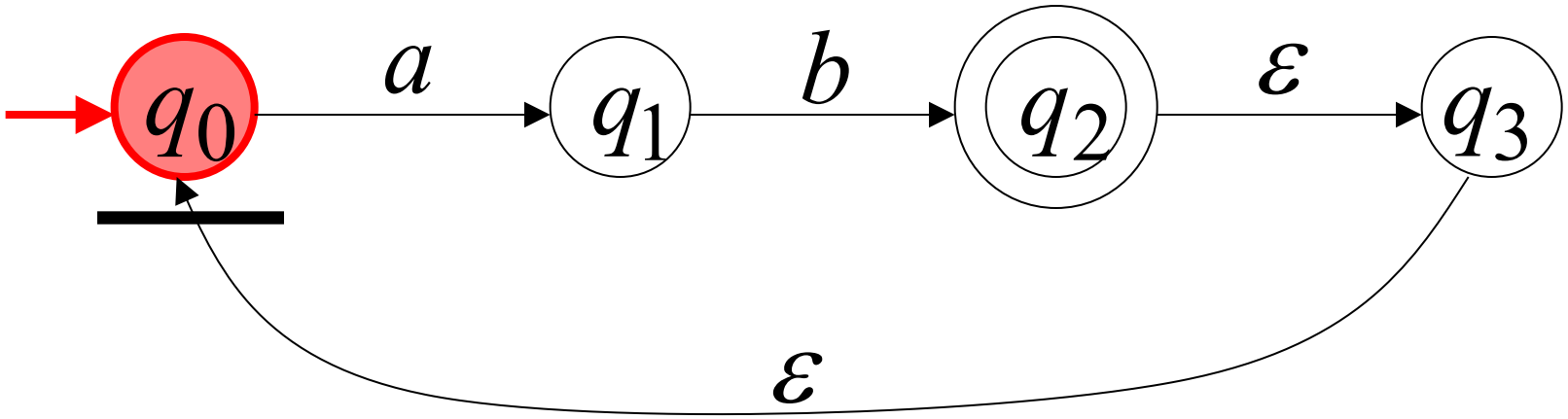

q_0
a
q_1
b
q_2
ε
q_3
ε

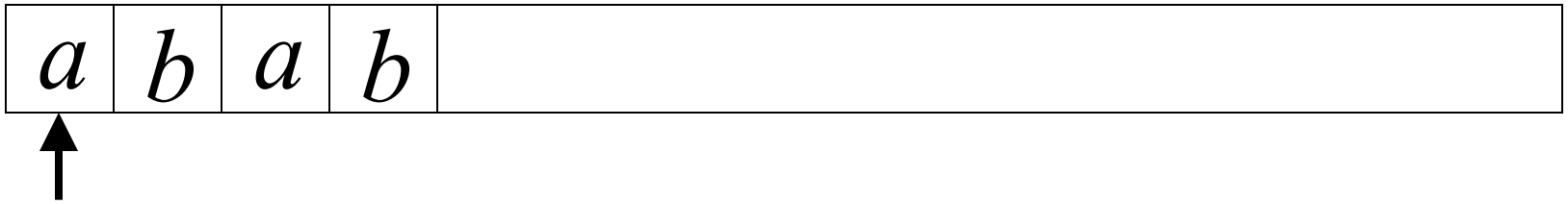

a
b

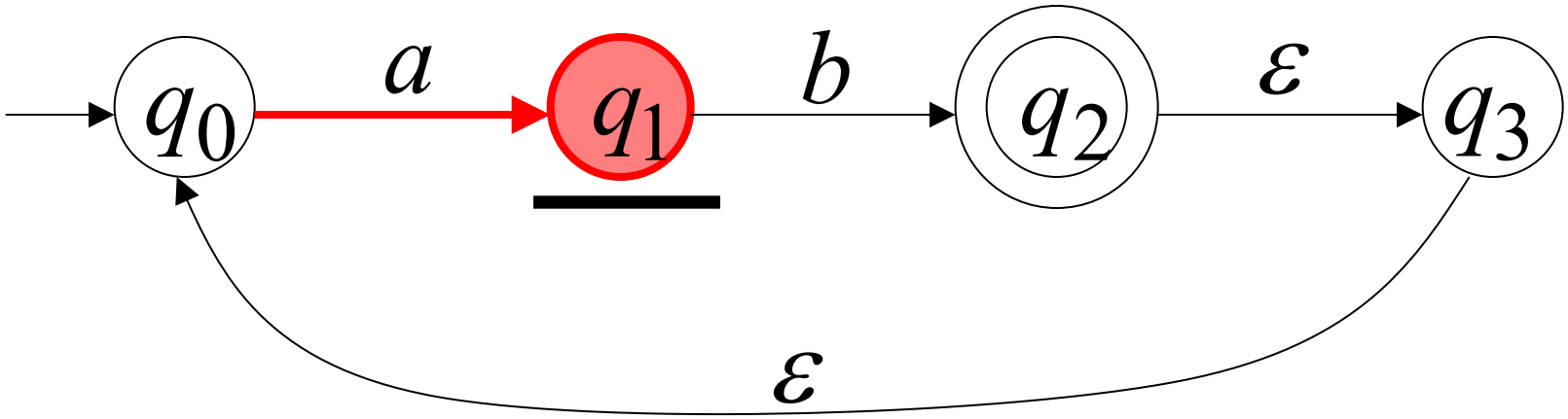

"accept"
q_0
a
q_1
b
q_2
ε
q_3
ε

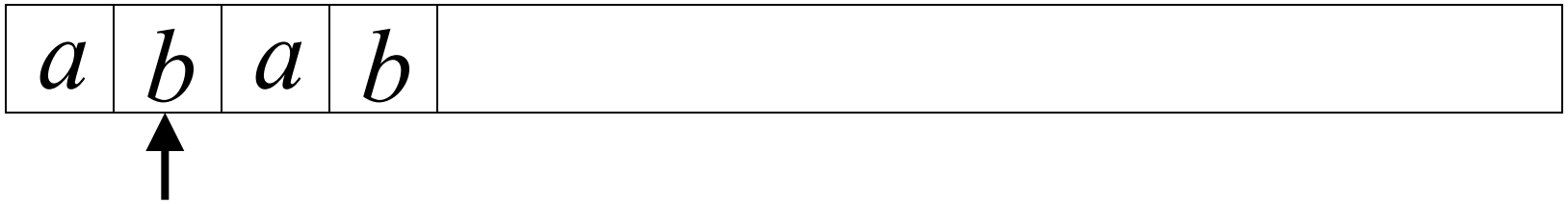
Another String

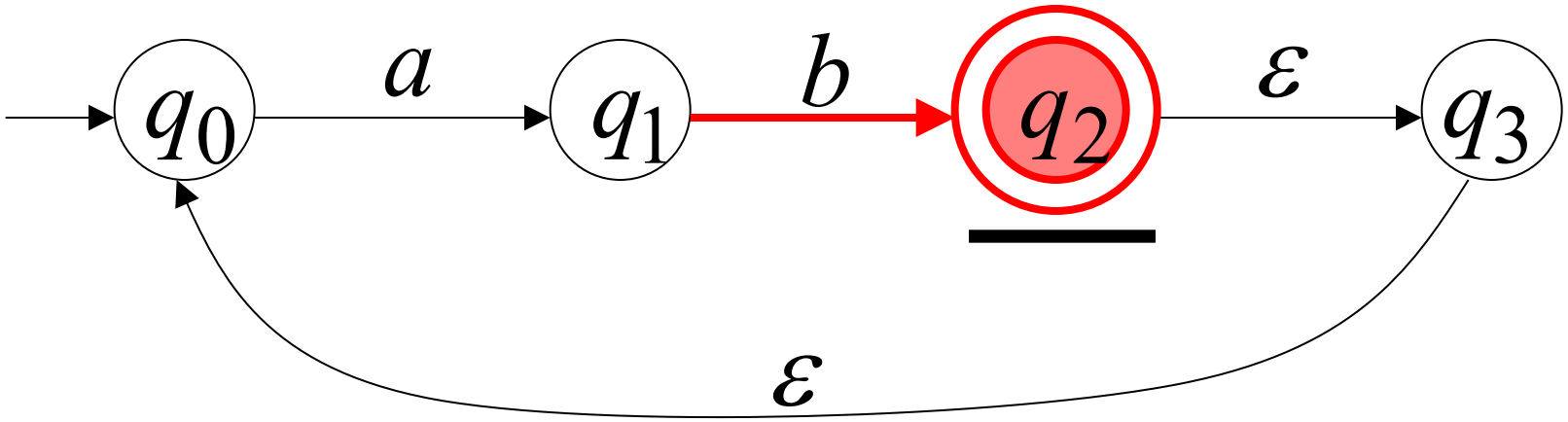
a	b	a	b	

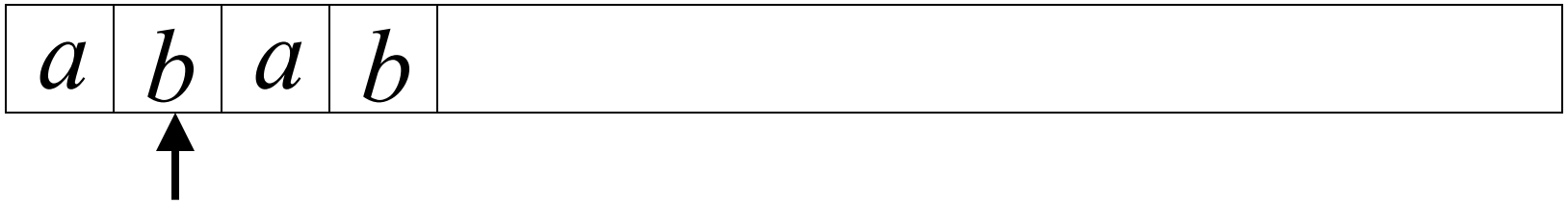


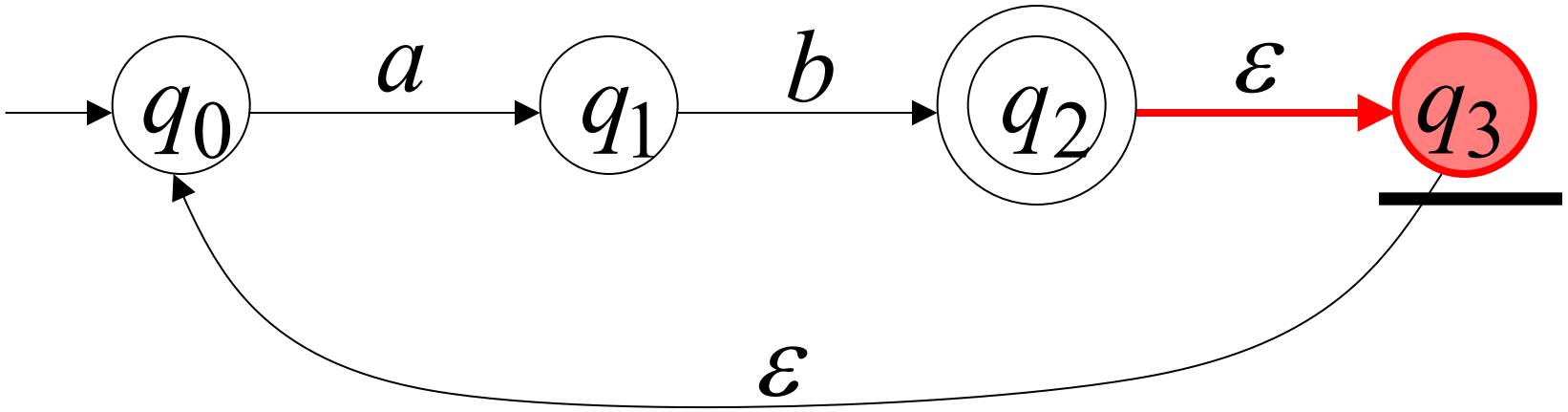

a
b
a
b

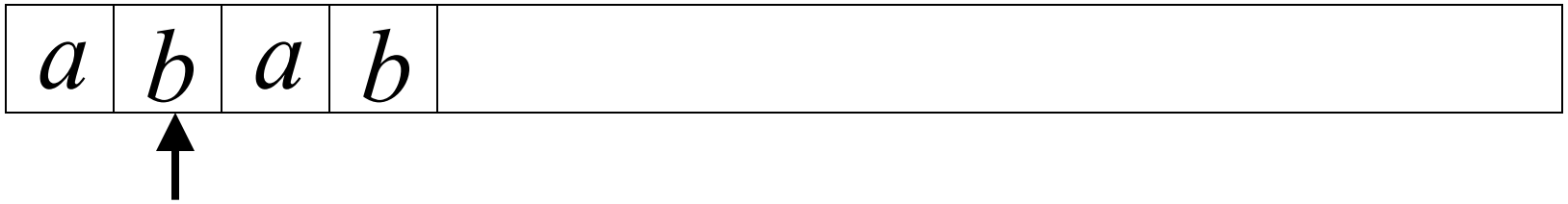

q_0
a
q_1
b
q_2
ε
q_3
ε

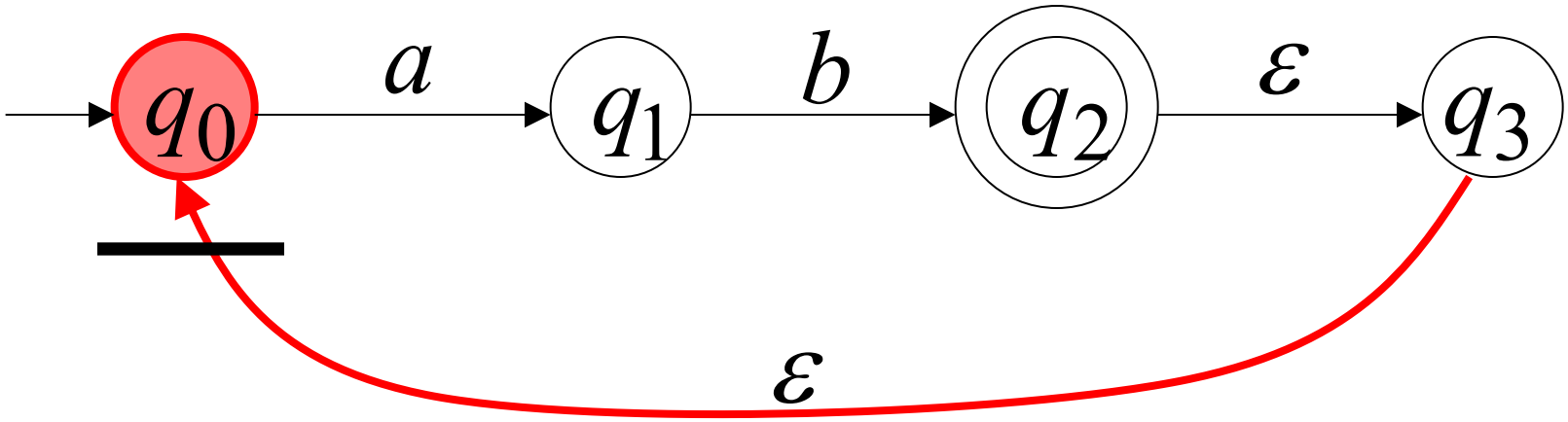

a
b
a
b

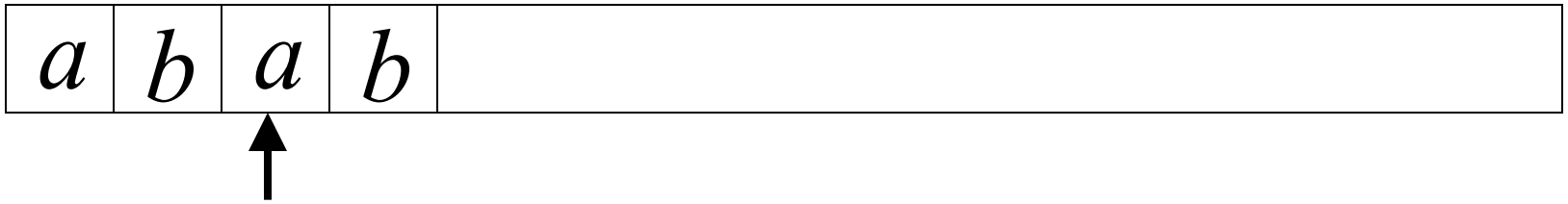

q_0
a
q_1
b
q_2
ε
q_3
ε

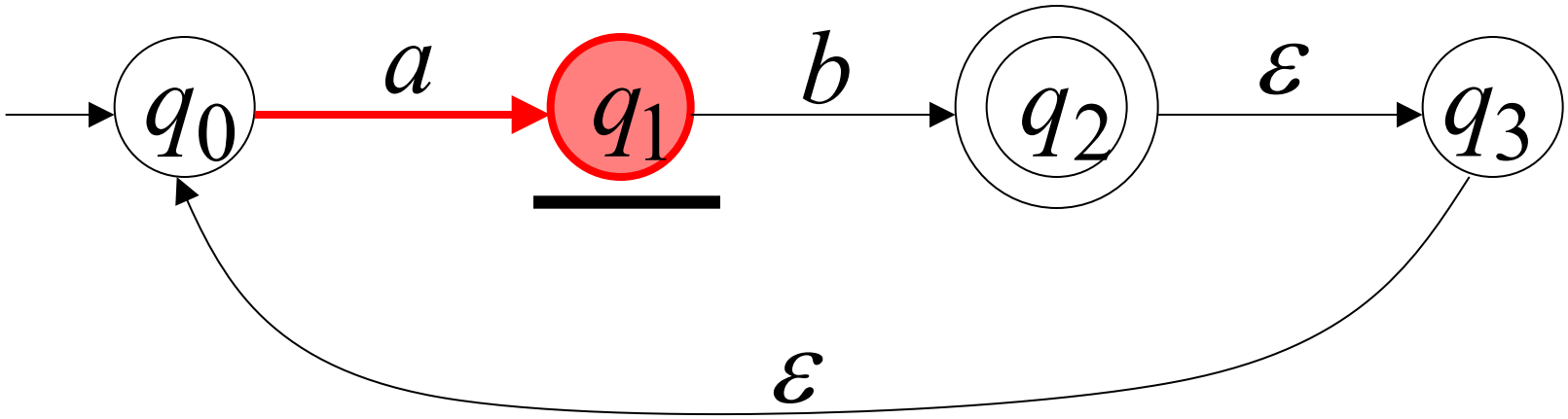

a
b
a
b

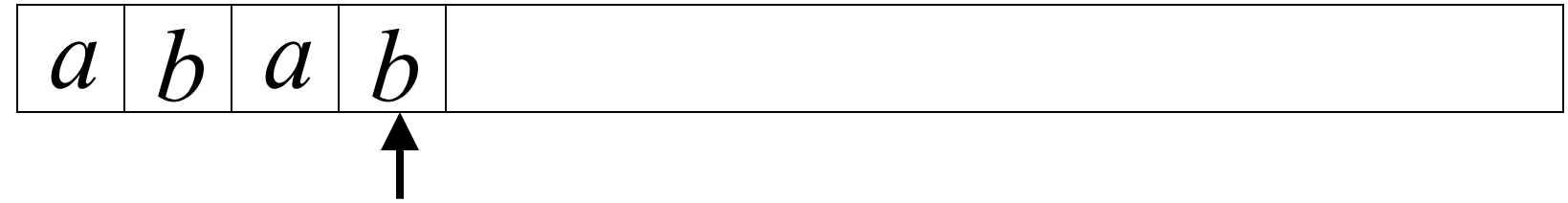

q_0
a
q_1
b
q_2
ε
q_3
ε

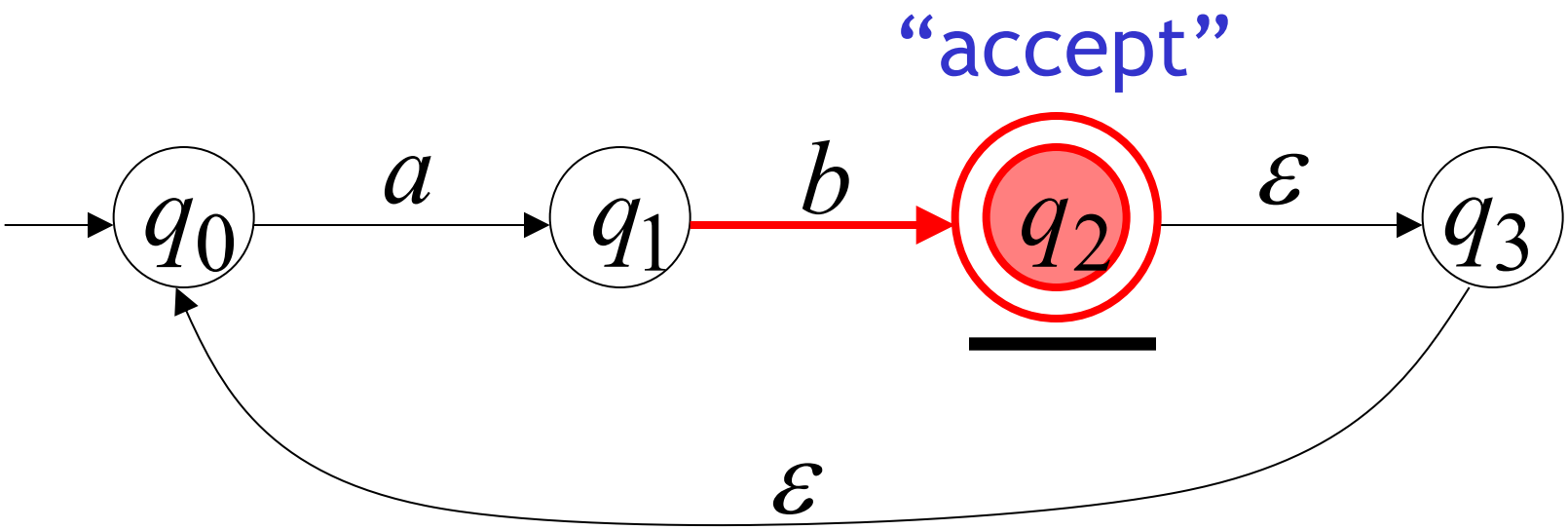

a
b
a
b


q_0
a
q_1
b
q_2
ε
q_3
ε


a
b
a
b

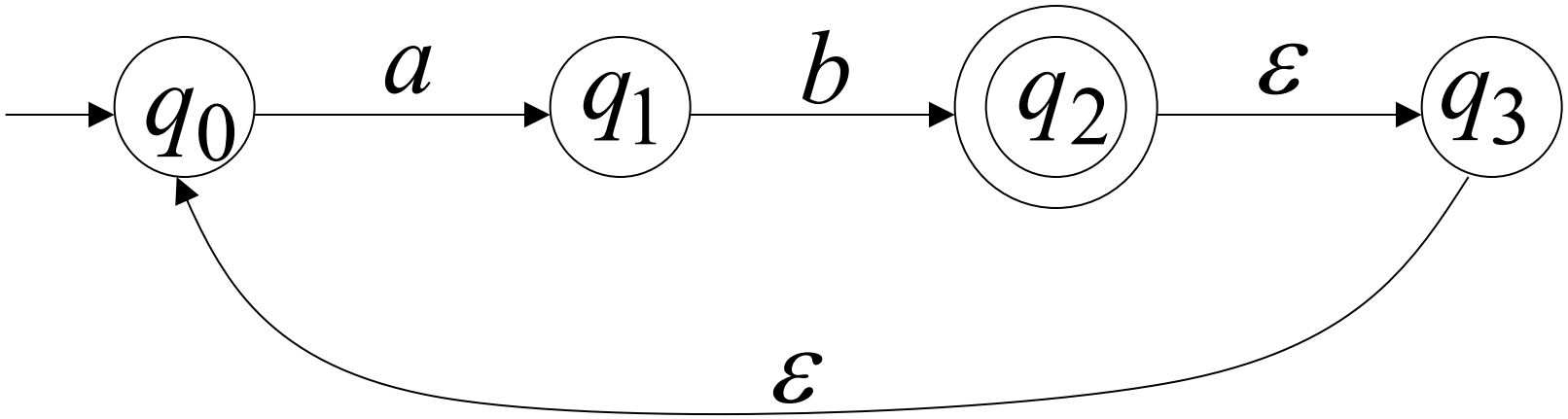

q_0
a
q_1
b
q_2
ε
q_3
ε


a
b
a
b

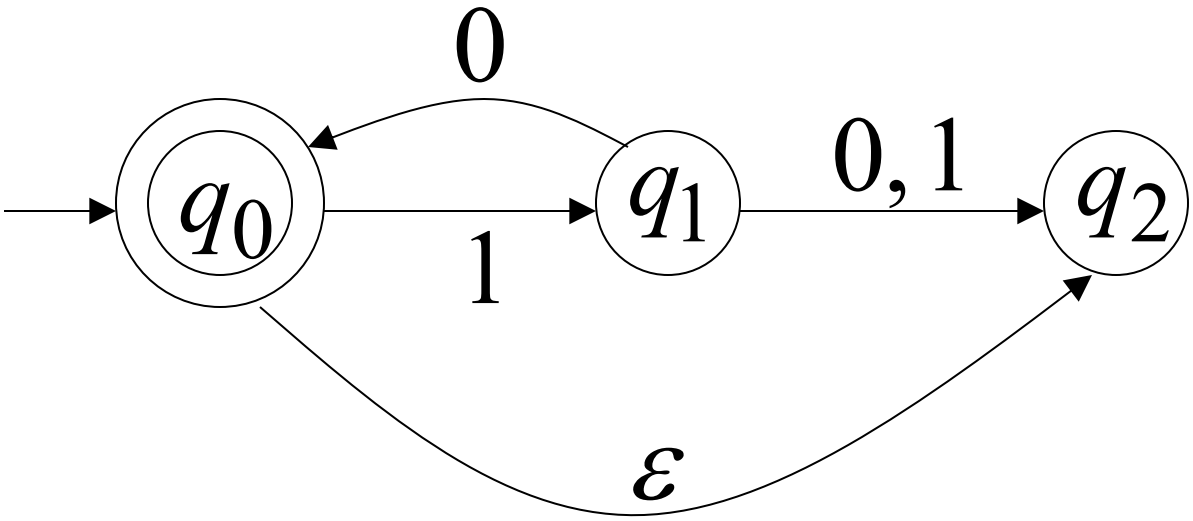

"accept"
q_0
q_1
q_2
q_3
a
b
ε
ε

Language recognized by this NFA:

$$L = \{ab,\ abab,\ ababab,\ ...\}$$
$$= \{ab\}\{ab\}^*$$



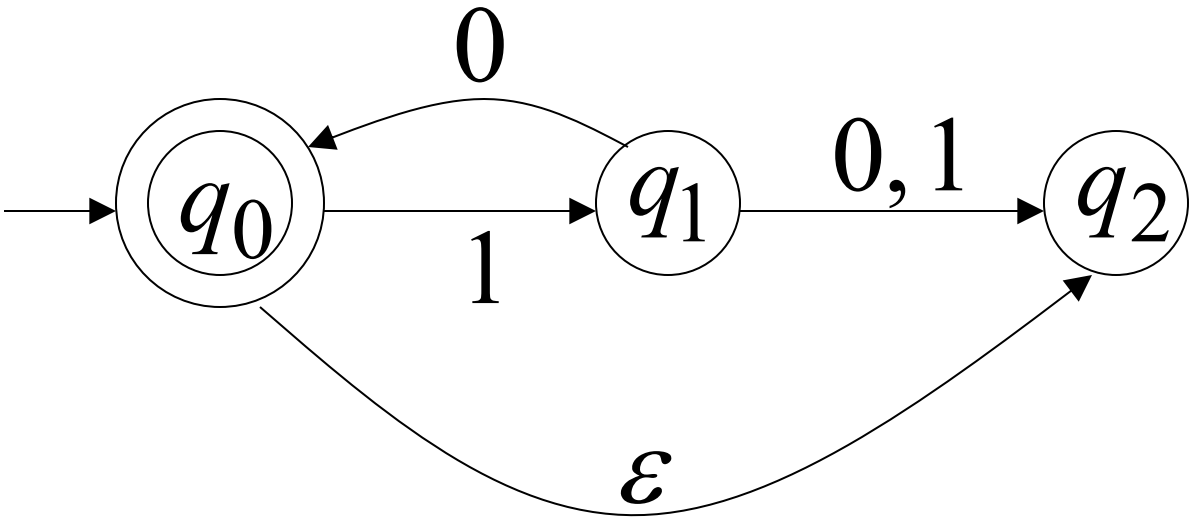
Another NFA Example



Language recognized

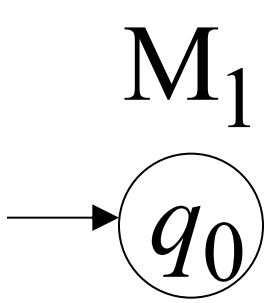
$$L = \{\varepsilon,\ 10,\ 1010,\ 101010,\ ...\}$$

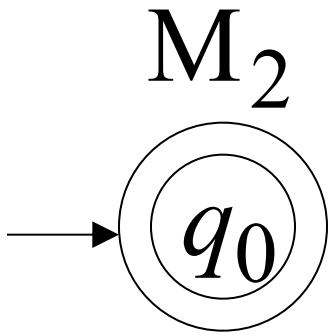
$$= \{10\}^*$$



(Redundant state, q2 can be removed)

Simple NFAs



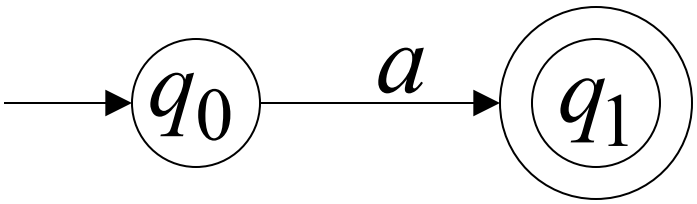


$L(M_1) = \{\}$

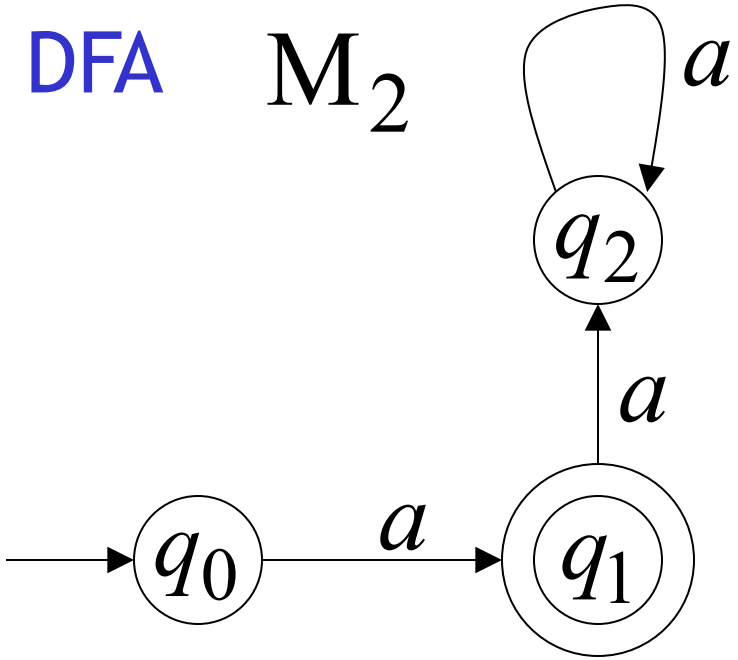
$L(M_2) = \{\varepsilon\}$

NFAs are interesting because we can express languages easier than DFAs

NFA M_1



DFA M_2



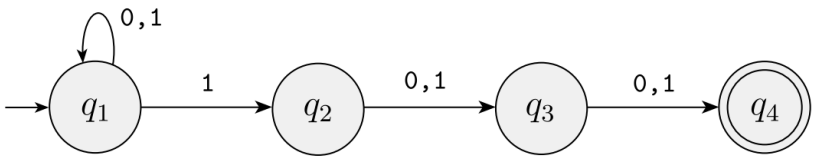
$$L(M_1) = \{a\}$$

$$L(M_2) = \{a\}$$

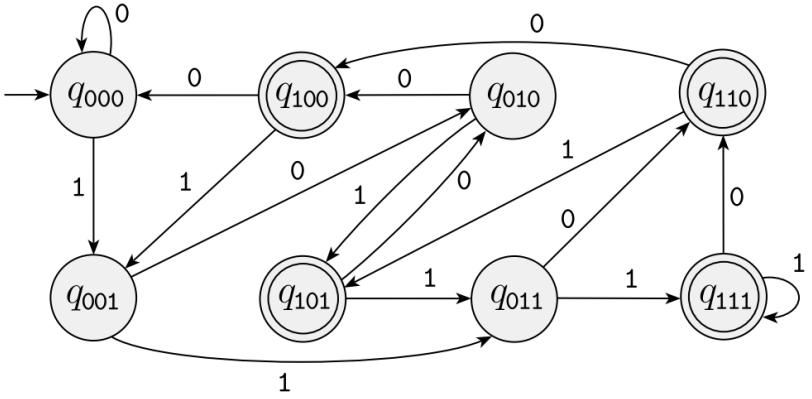
Another Example:

Let A be the language consisting of all strings over {0,1} containing a 1 in the third position from the end (e.g., 000100 is in A but 0011 is not).

NFA recognizing A



DFA recognizing A



Formal Definition of NFAs

$$M = (Q,\ \Sigma,\ \delta,\ q_0,\ F)$$

Q: Set of states, i.e. $\{q_0, q_1, q_2\}$

Σ: Input alphabet, i.e. $\{a, b\}$ $\quad\quad$ $\varepsilon \notin \Sigma$

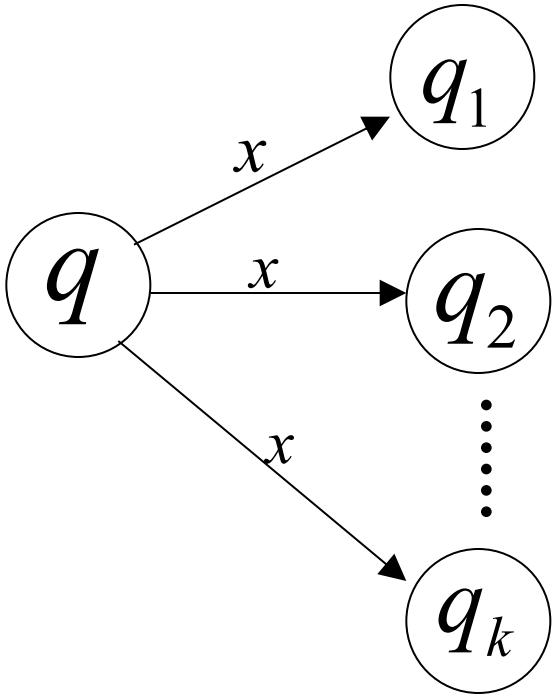
δ: Transition function

q_0: Initial state

F: Accepting states

Transition Function δ

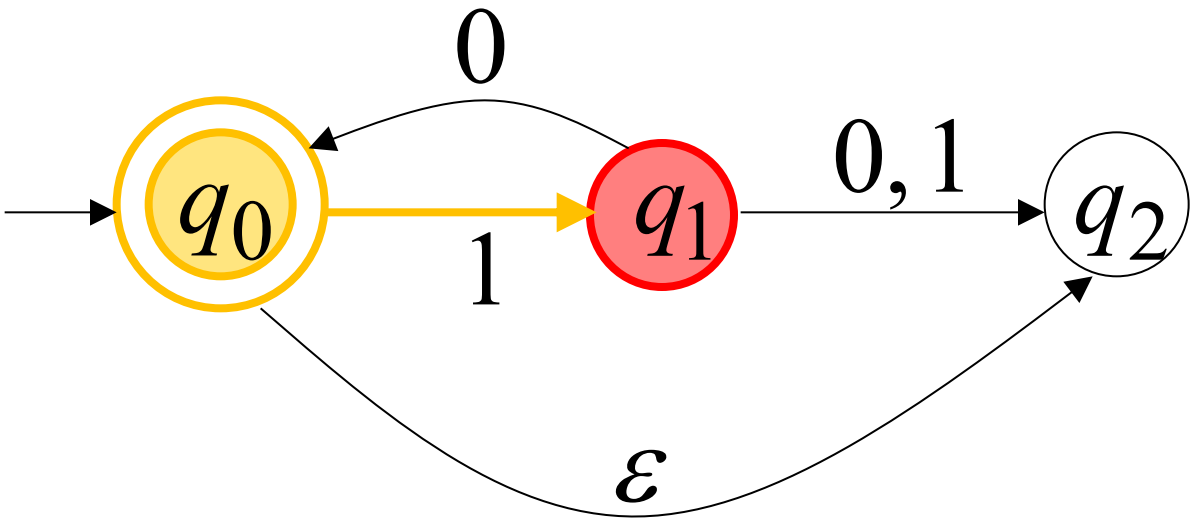
$$\delta(q, x) = \{q_1, q_2, \ldots, q_k\}$$



resulting states reached by following one transition with input symbol x

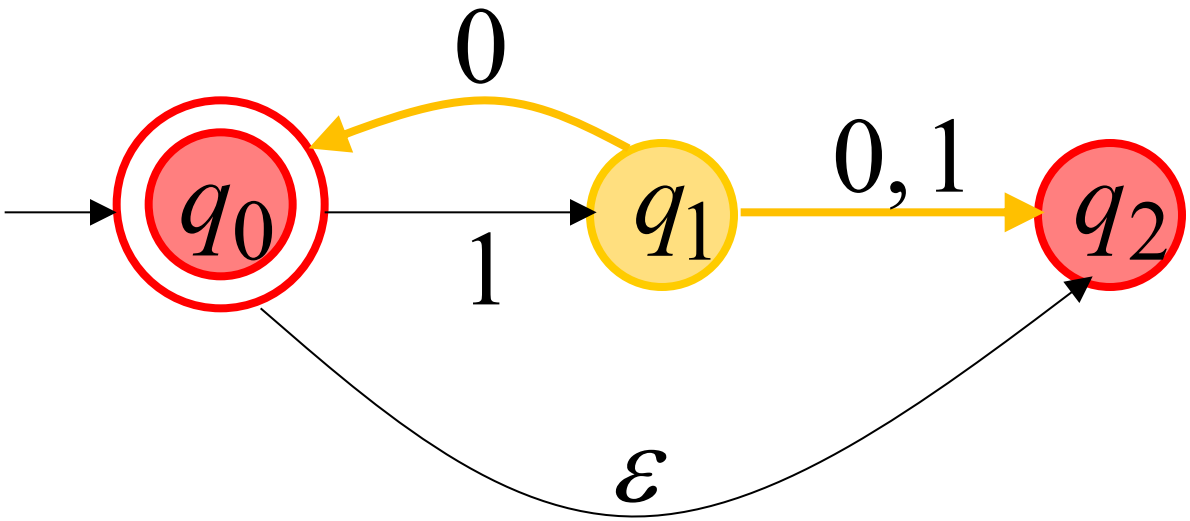
States reachable from q_0 scanning 1

$$\delta(q_0, 1) = \{q_1\}$$



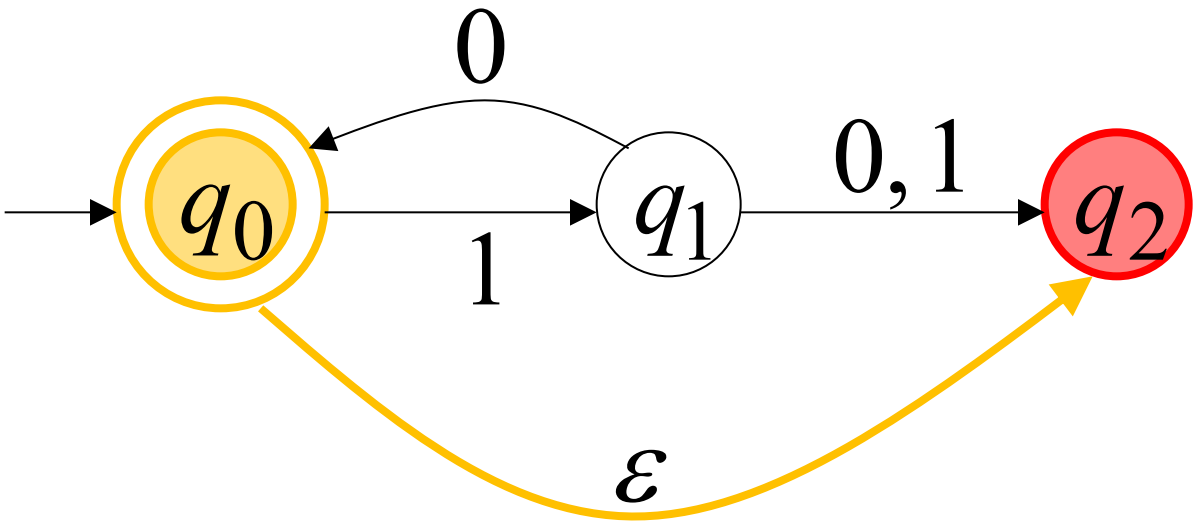
States reachable from q_1 scanning 0

$$\delta(q_1,0) = \{q_0, q_2\}$$



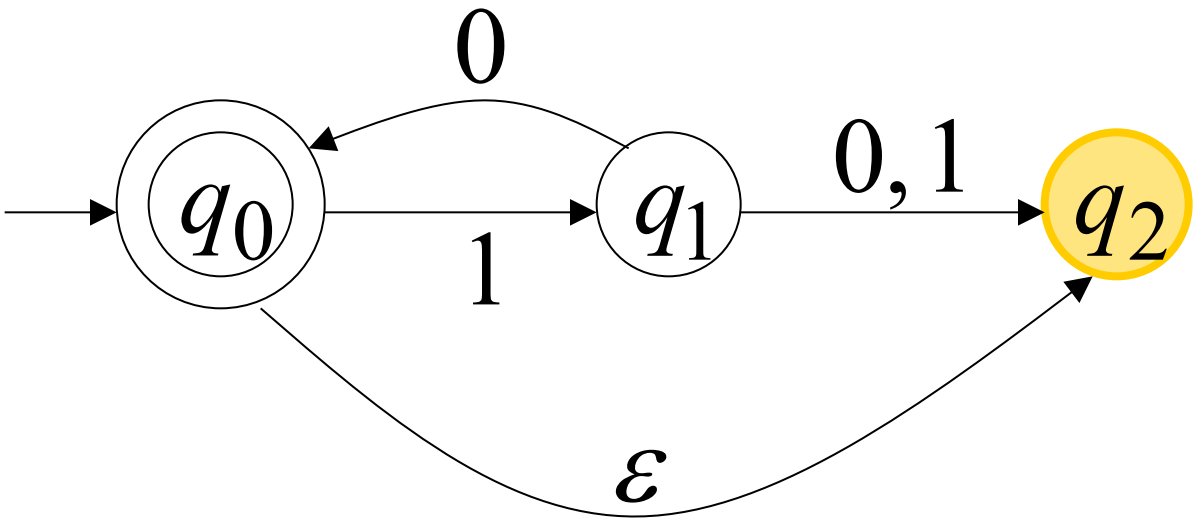
States reachable from q_0 with one transition scanning no input symbol

$$\delta(q_0, \varepsilon) = \{q_2\}$$



States reachable from q_2 scanning 1

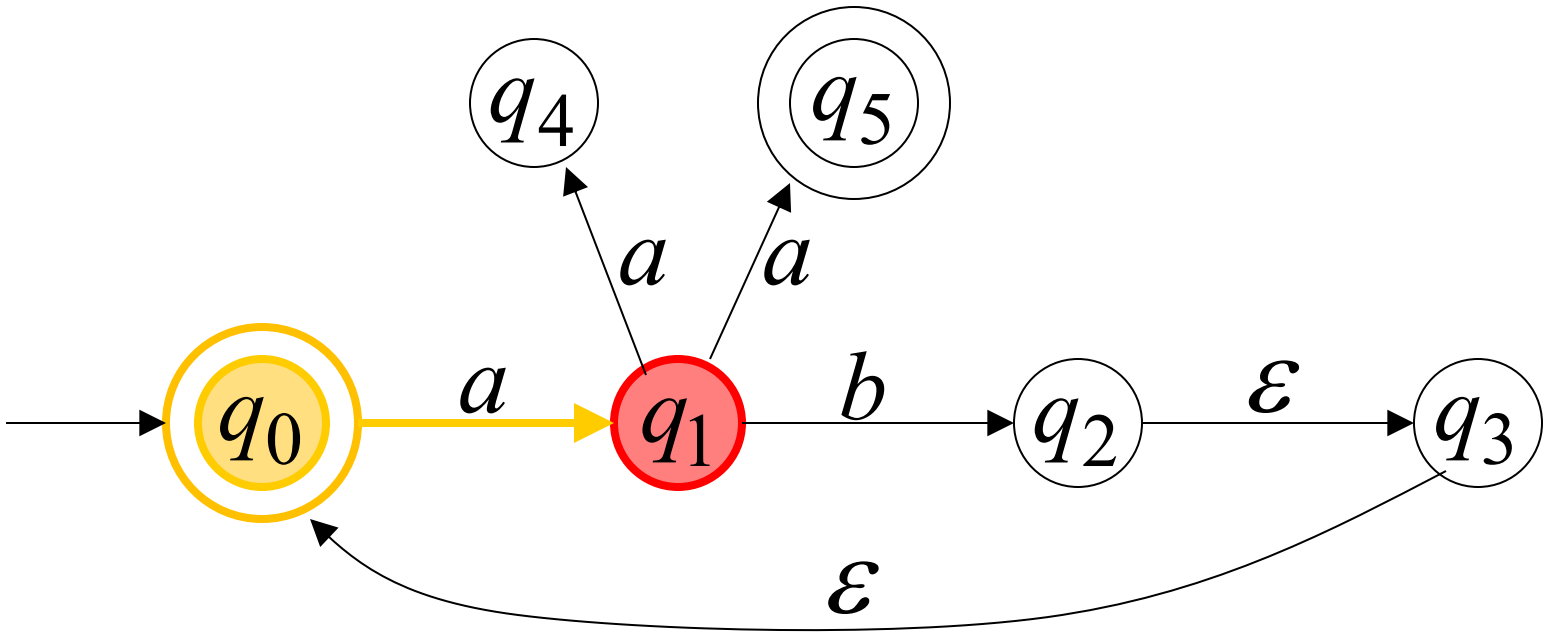
$$\delta(q_2, 1) = \varnothing$$



Extended Transition Function δ^*

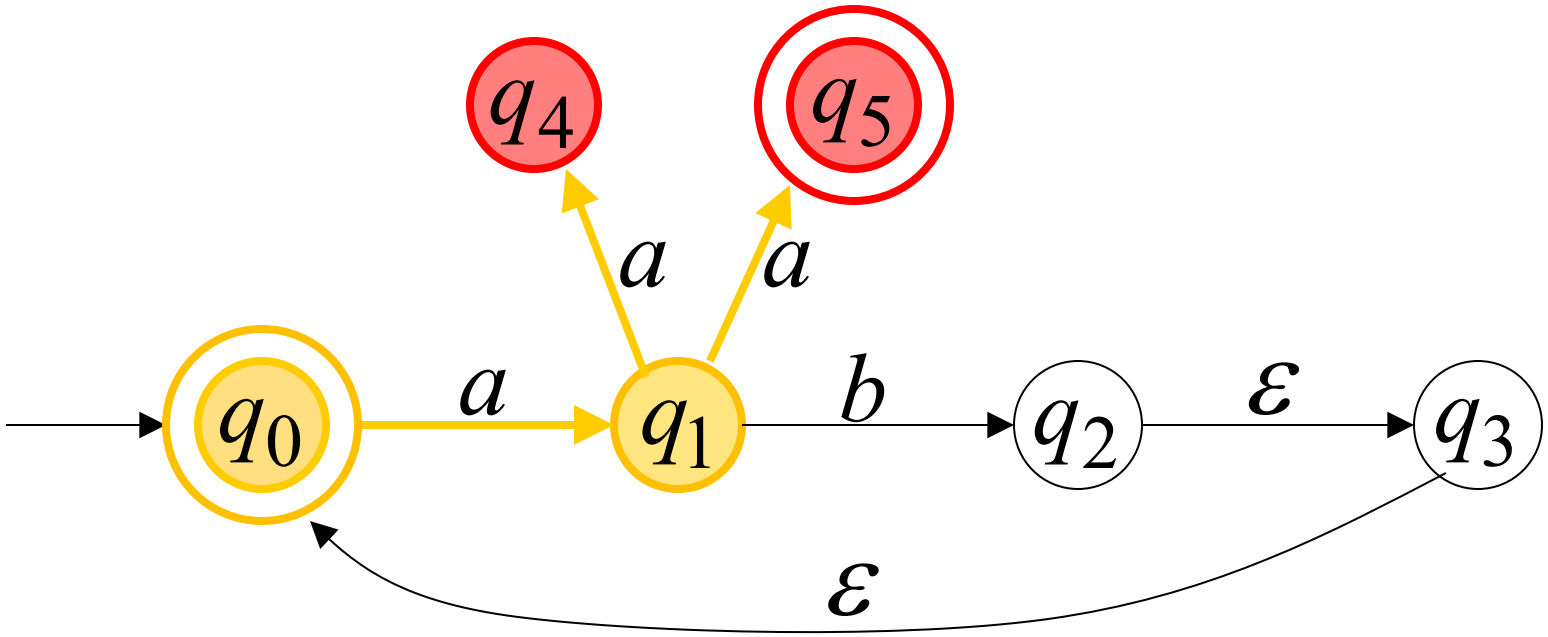
Similar with δ but applied on strings

$$\delta^*(q_0, a) = \{q_1\}$$



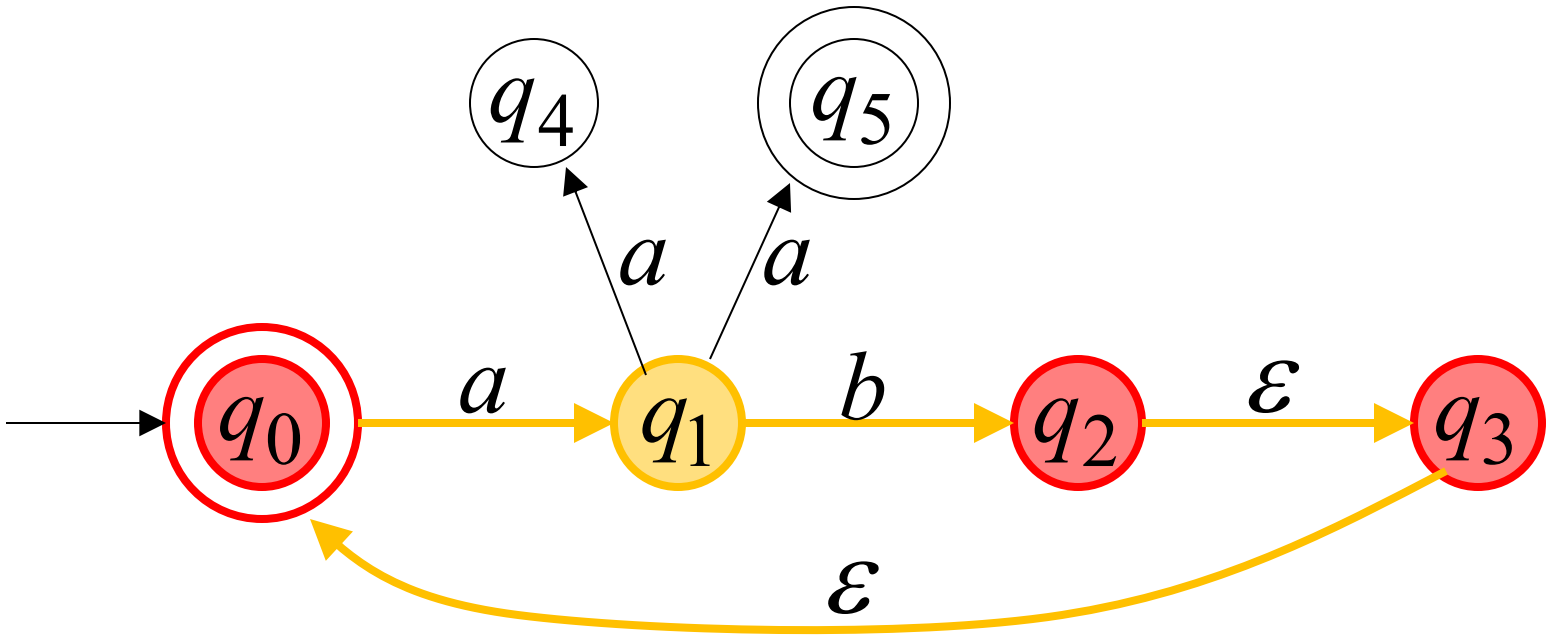
States reachable from q_0 scanning aa

$$\delta^*(q_0, aa) = \{q_4, q_5\}$$



States reachable from q_0 scanning ab

$$\delta^*(q_0, ab) = \{q_2, q_3, q_0\}$$



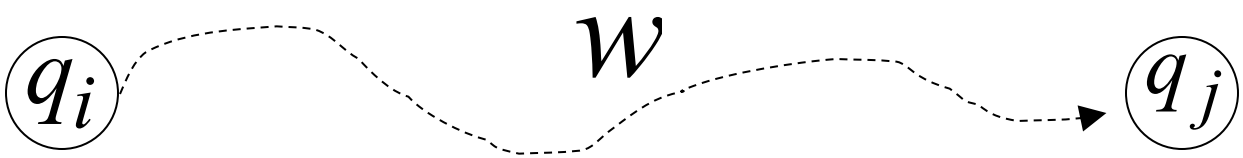
Special case:

for any state q

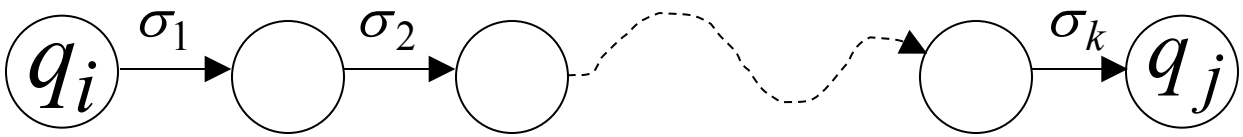
$$q \in \delta^*(q, \varepsilon)$$

In general

$q_j \in \delta^*(q_i, w)$: there is a walk from q_i to q_j with string w



$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$



The Language of an NFA M

The language recognized by M is:

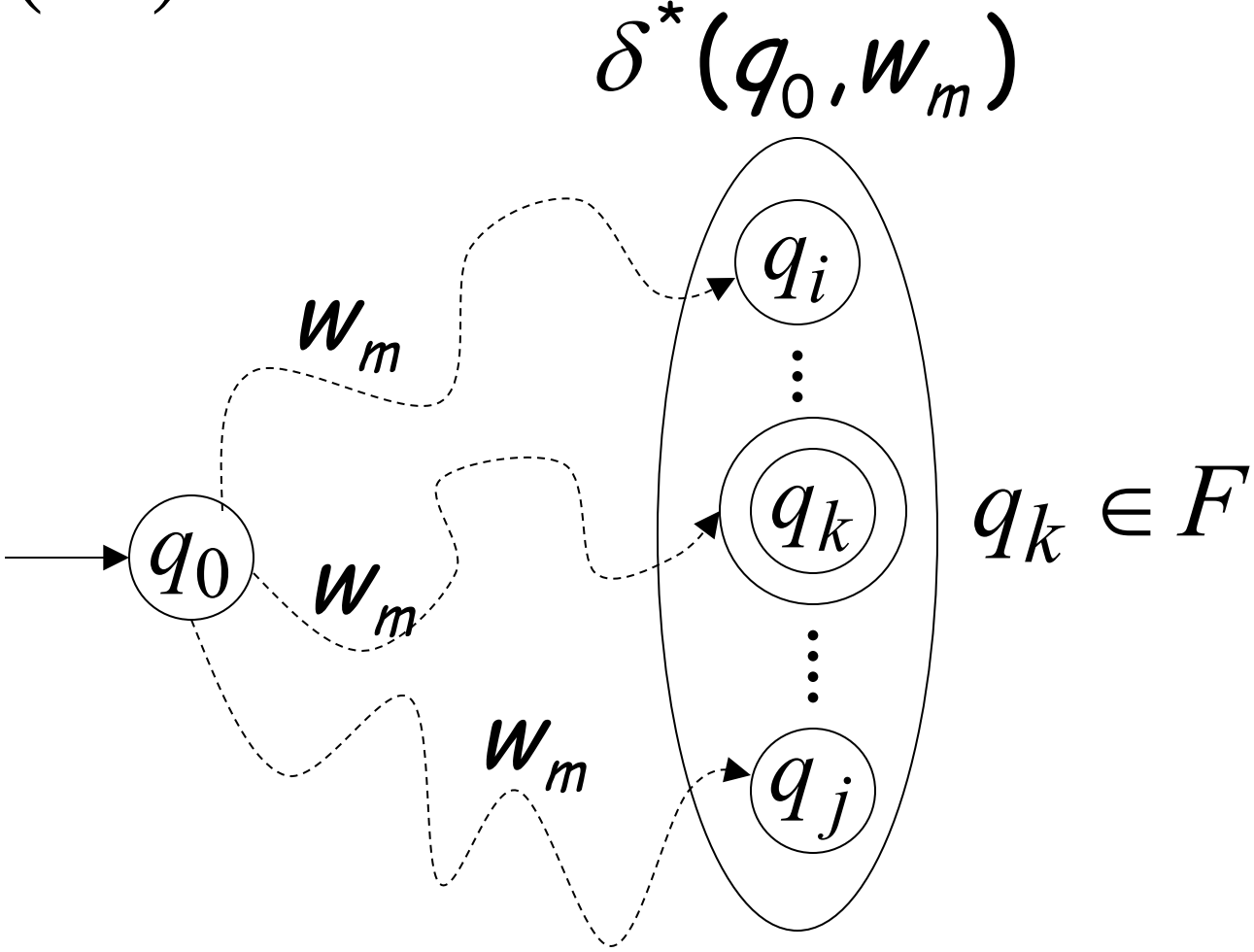
$$L(M) = \{w_1, w_2, \ldots, w_n\}$$

Where for each w_m

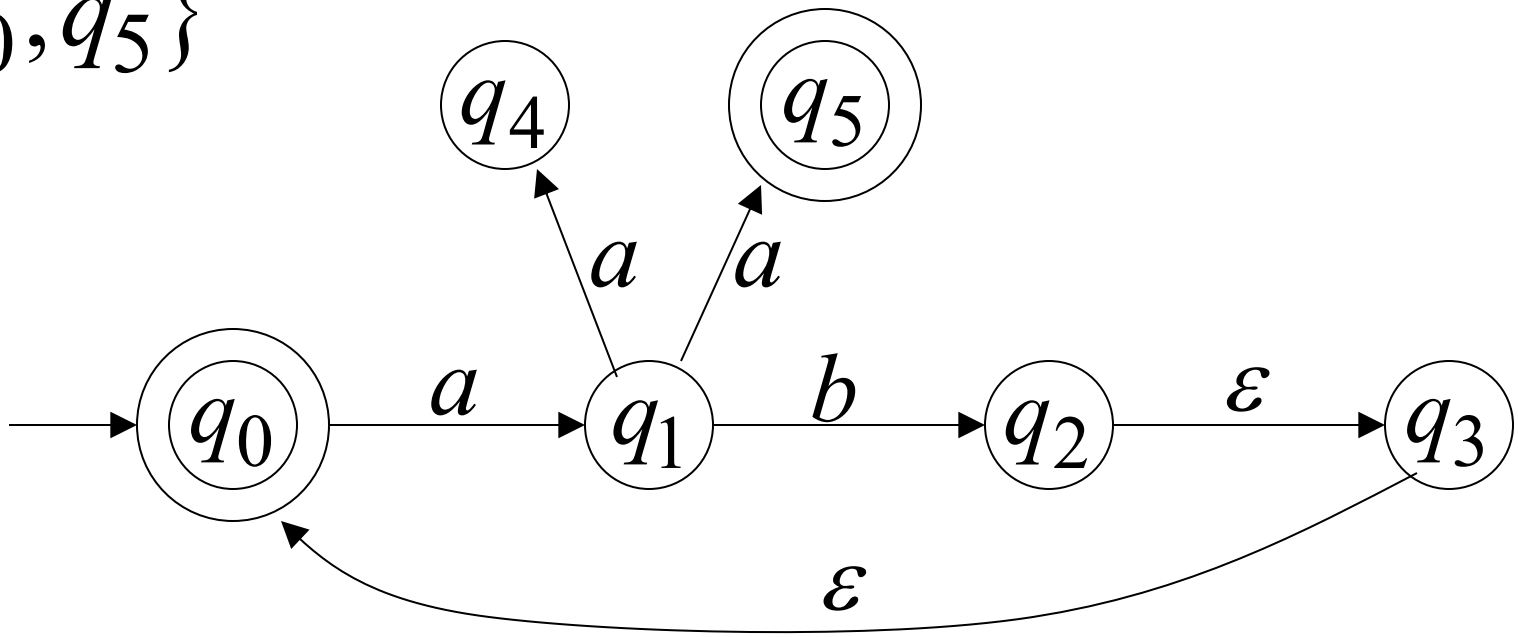
$$\delta^*(q_0, w_m) = \{q_i, \ldots, q_k, \ldots, q_j\}$$

and there is some $q_k \in F$ (accepting state)

$w_m \in L(M)$



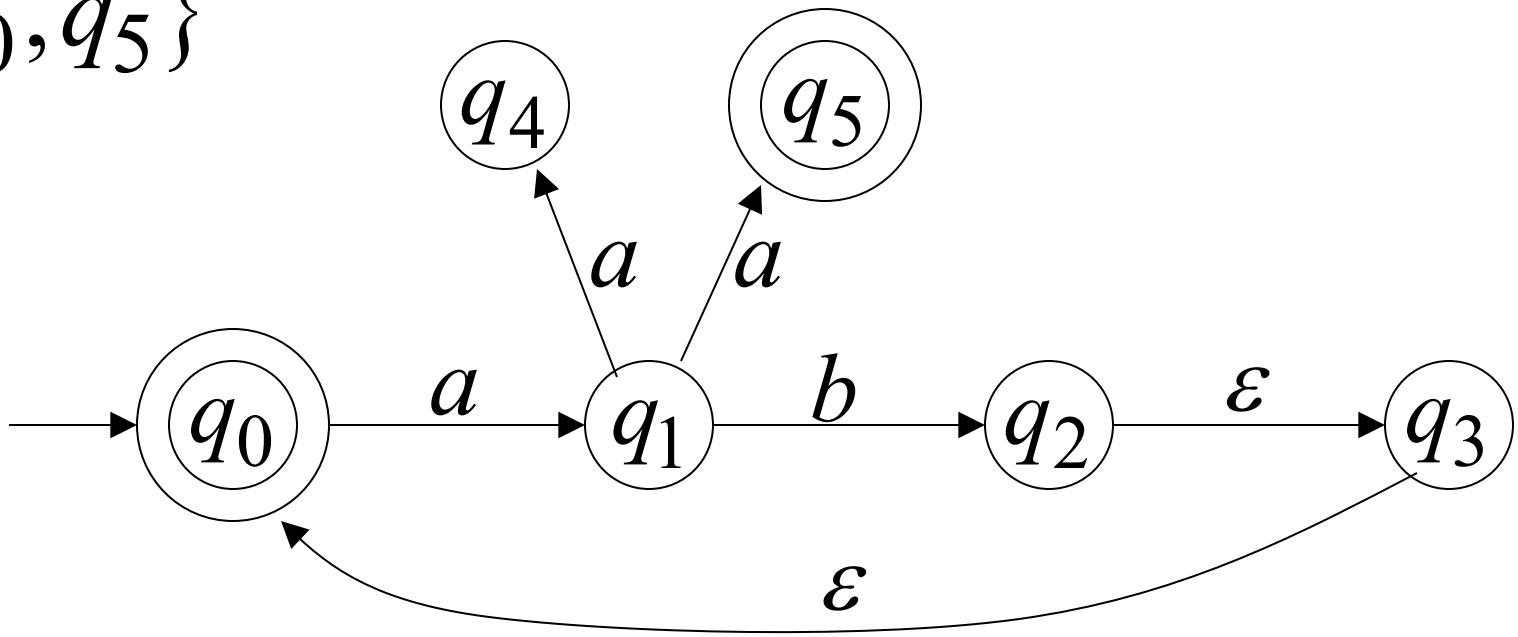
$$F = \{q_0, q_5\}$$



$$\delta^*(q_0, aa) = \{q_4, \underline{q_5}\} \Rightarrow aa \in L(M)$$

$$\underline{q_5} \in F$$

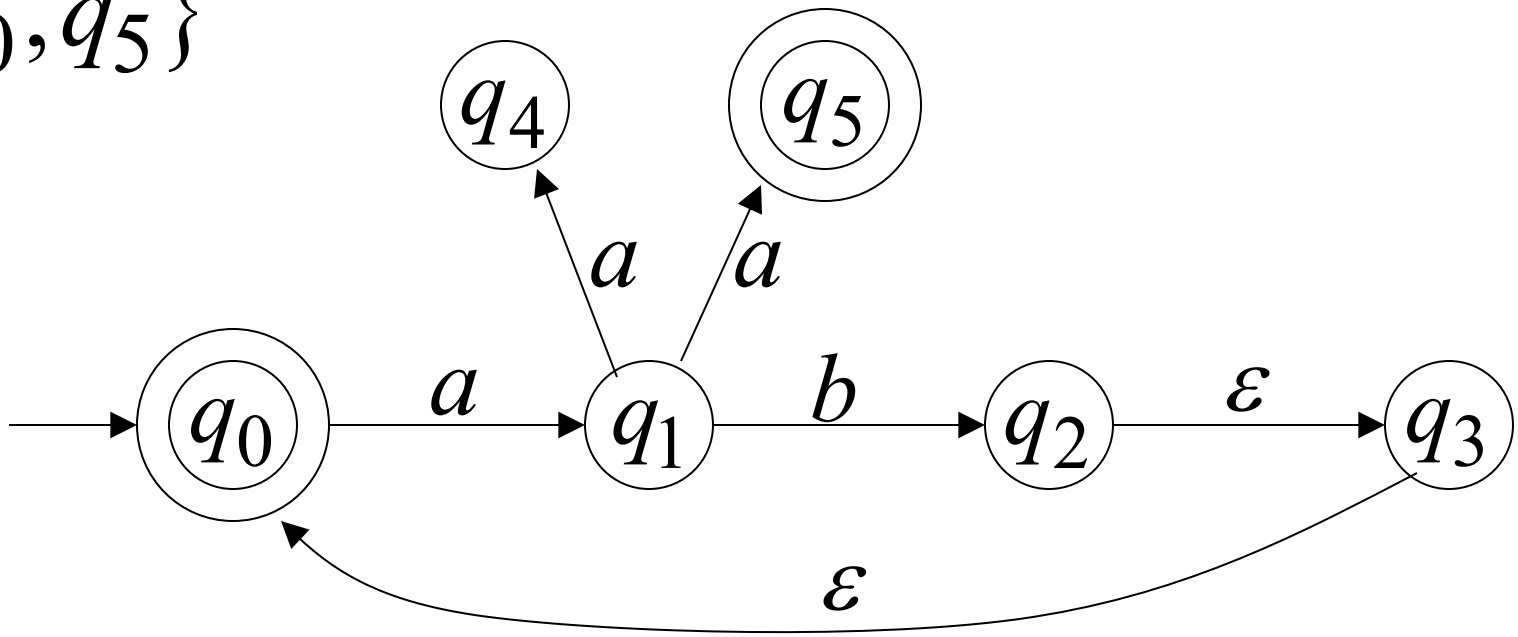
$$F = \{q_0, q_5\}$$



$$\delta^*(q_0, ab) = \{q_2, q_3, \underline{q_0}\} \Rightarrow ab \in L(M)$$

$$\underline{q_0} \in F$$

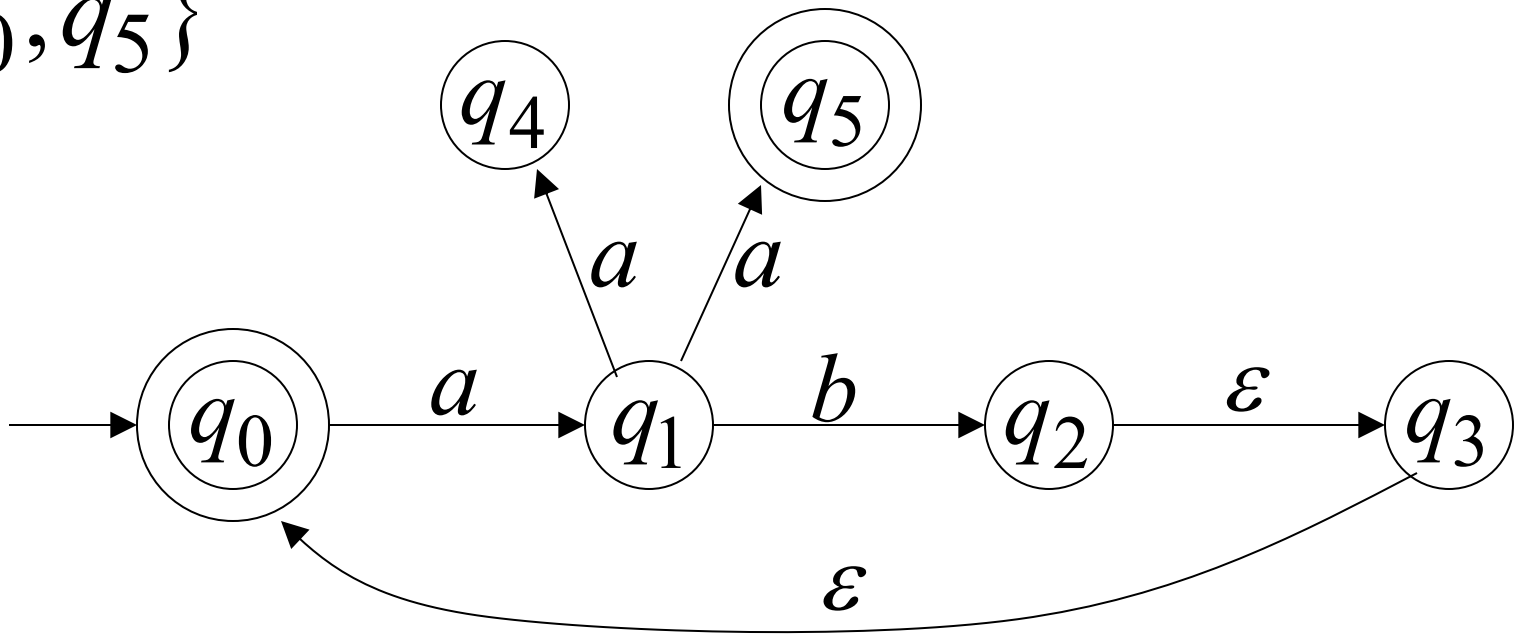
$$F = \{q_0, q_5\}$$



$$\delta^*(q_0, abaa) = \{q_4, \underline{q_5}\} \Rightarrow abaa \in L(M)$$

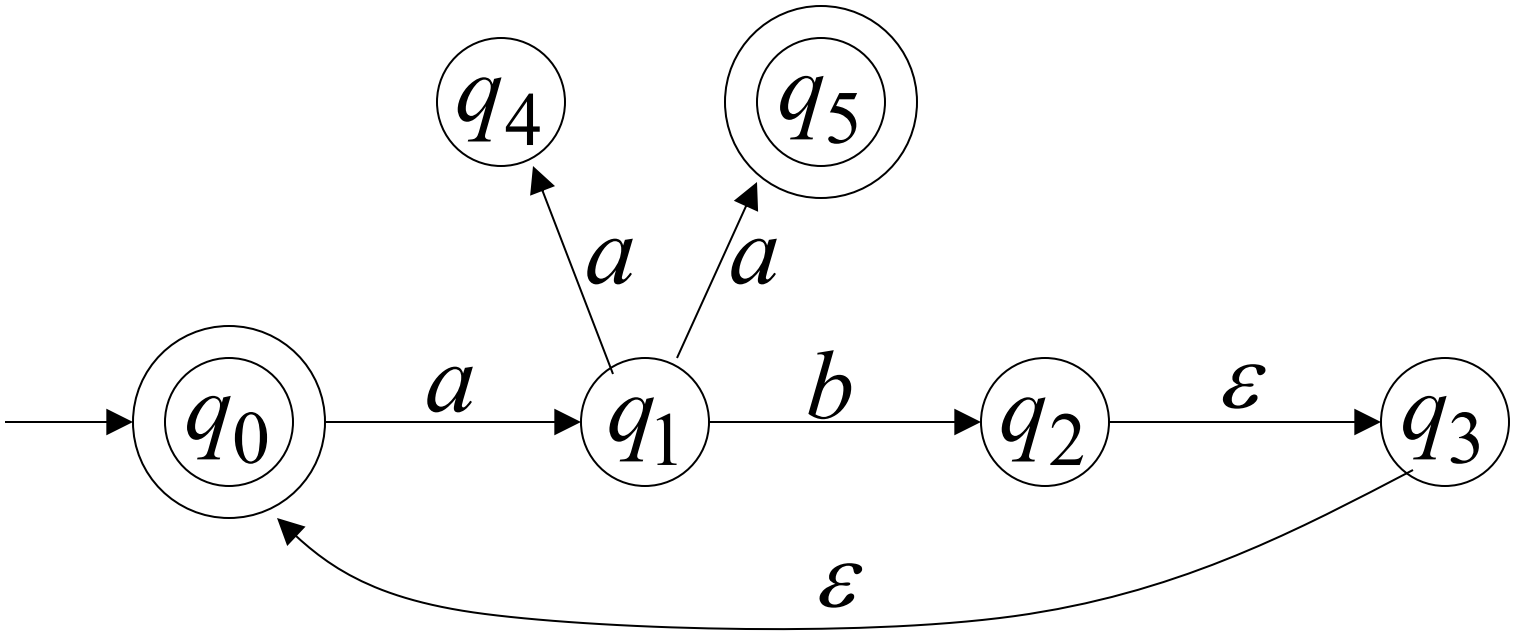
$$\searrow \in F$$

$F = \{q_0, q_5\}$



$$\delta^*(q_0, aba) = \{q_1\} \Rightarrow aba \notin L(M)$$

$$\notin F$$



$$L(M) = \{ab\}^* \cup \{ab\}^* \{aa\}$$

Like DFAs, NFAs recognize the Regular Languages!

Equivalence of Machines

Definition:

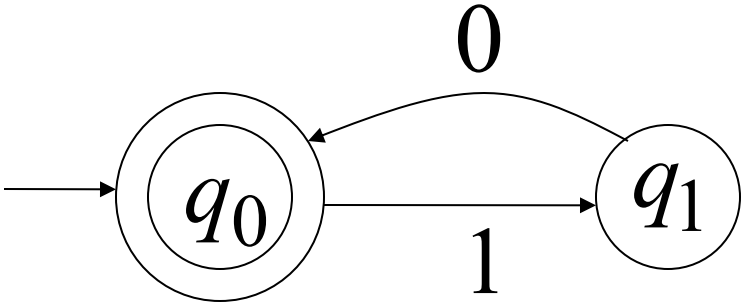
Machine M_1 is equivalent to machine M_2

iff $L(M_1) = L(M_2)$

Example of equivalent machines

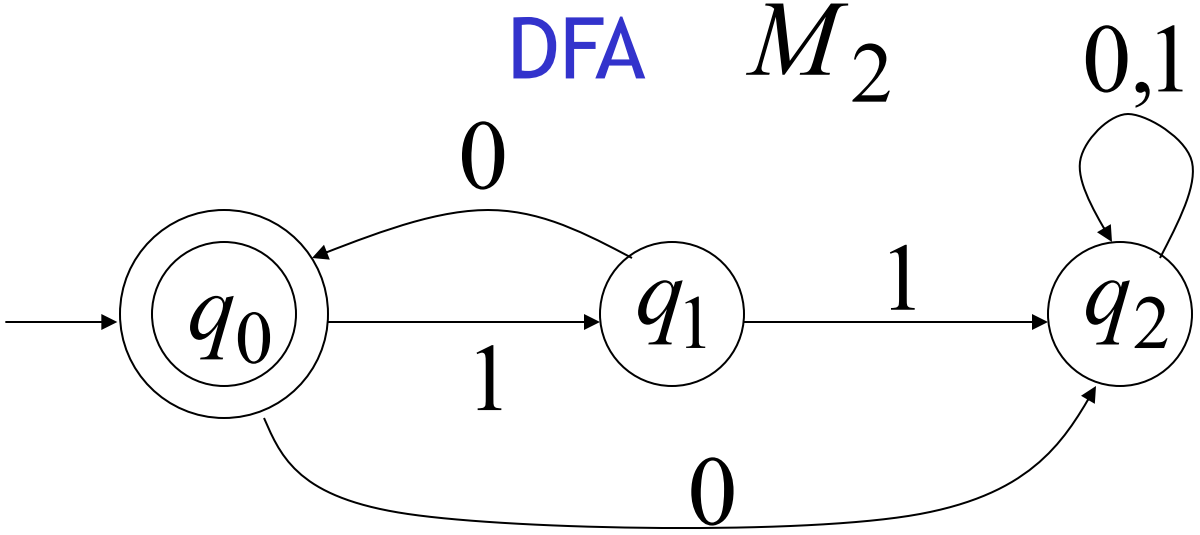
$L(M_1) = \{10\}^*$

NFA M_1



$L(M_2) = \{10\}^*$

DFA M_2



Theorem:

$$\left\{\begin{array}{l}\text{Languages}\\ \text{recognized}\\ \text{by NFAs}\end{array}\right\} = \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

Languages recognized by DFAs

NFAs and DFAs have the same computation power, namely, they recognize the same set of languages.

Proof: We need to show

$$\left\{\begin{array}{l}\text{Languages}\\ \text{recognized}\\ \text{by NFAs}\end{array}\right\} \supseteq \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

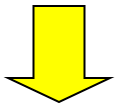
AND

$$\left\{\begin{array}{l}\text{Languages}\\ \text{recognized}\\ \text{by NFAs}\end{array}\right\} \subseteq \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

Proof-Step 1

$$\left\{\begin{array}{l}\text{Languages}\\ \text{recognized}\\ \text{by NFAs}\end{array}\right\} \supseteq \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

Every DFA is trivially a NFA

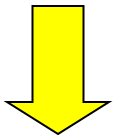


Any language L recognized by a DFA is also recognized by a NFA

Proof-Step 2

$$\left\{\begin{array}{l}\text{Languages}\\ \text{recognized}\\ \text{by NFAs}\end{array}\right\} \subseteq \left\{\begin{array}{l}\text{Regular}\\ \text{Languages}\end{array}\right\}$$

Any NFA can be converted to an equivalent DFA



Any language L recognized by a NFA is also recognized by a DFA

General NFA->DFA Conversion Procedure

Input: an NFA M

Output: an equivalent DFA M'

with $L(M) = L(M')$

The NFA has states $q_0, q_1, q_2, \cdots$

The DFA has states from the power set

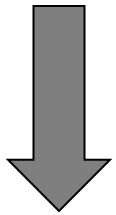
$$\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}, \{q_1, q_2, q_3\}, \ldots.$$

Conversion Procedure Steps

step

1. Initial state of NFA: q_0

$$\delta^*(q_0, \varepsilon) = \{q_0, \ldots\}$$

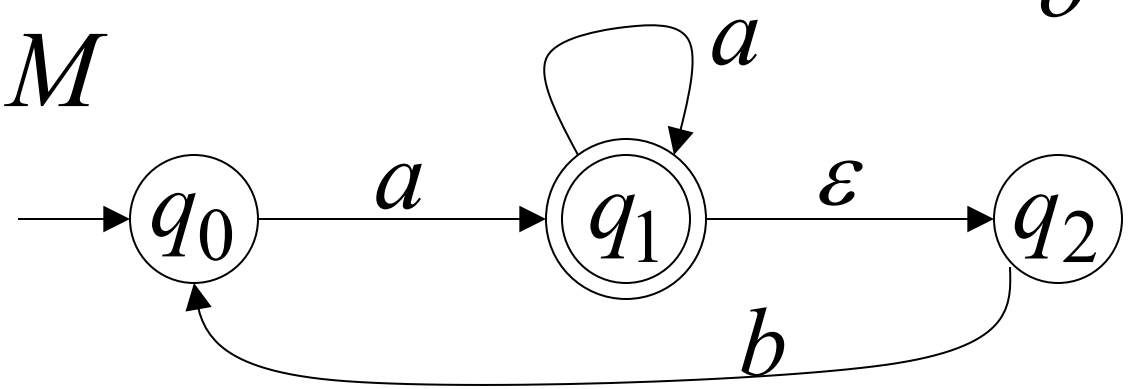


Initial state of DFA: $\{q_0, \ldots\}$

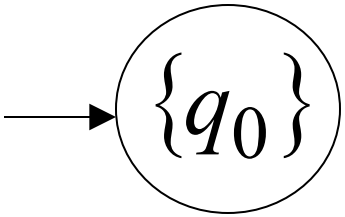
Example

$$\delta^*(q_0, \varepsilon) = \{q_0\}$$

NFA M



DFA M'



step

2. For every DFA's state $\{q_i, q_j, \ldots, q_m\}$

compute in the NFA

$$\left.\begin{array}{r} \delta^*(q_i, a) \\ \cup \delta^*(q_j, a) \\ \ldots \\ \cup \delta^*(q_m, a) \end{array}\right\} = \{q'_k, q'_l, \ldots, q'_n\}$$

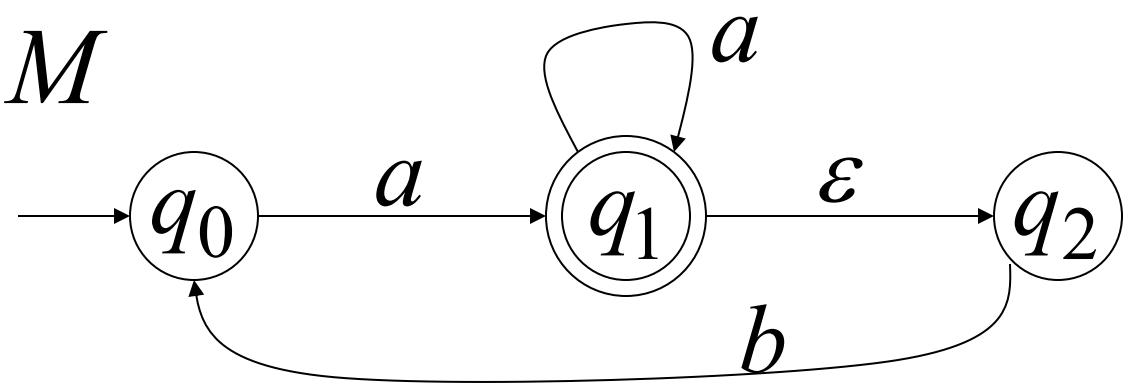
Union

add transition to DFA

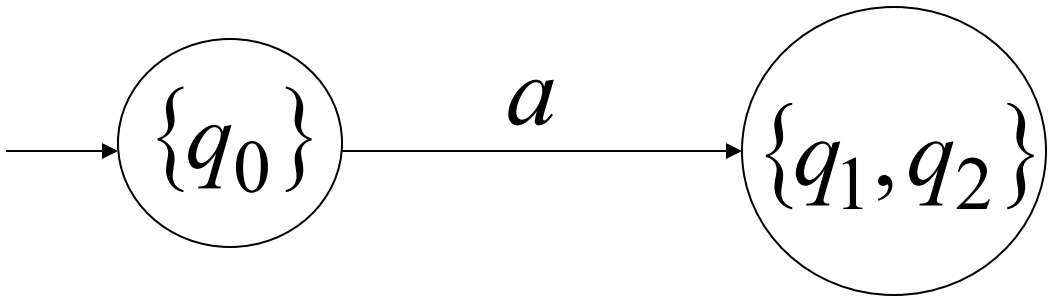
$$\delta(\{q_i, q_j, \ldots, q_m\},\ a) = \{q'_k, q'_l, \ldots, q'_n\}$$

Example $\delta^*(q_0, a) = \{q_1, q_2\}$

NFA M



DFA M'



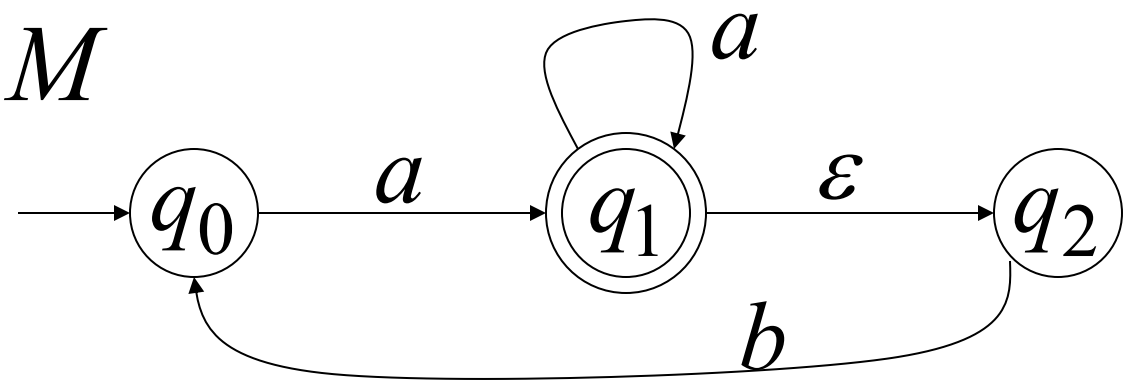
$$\delta(\{q_0\}, a) = \{q_1, q_2\}$$

step

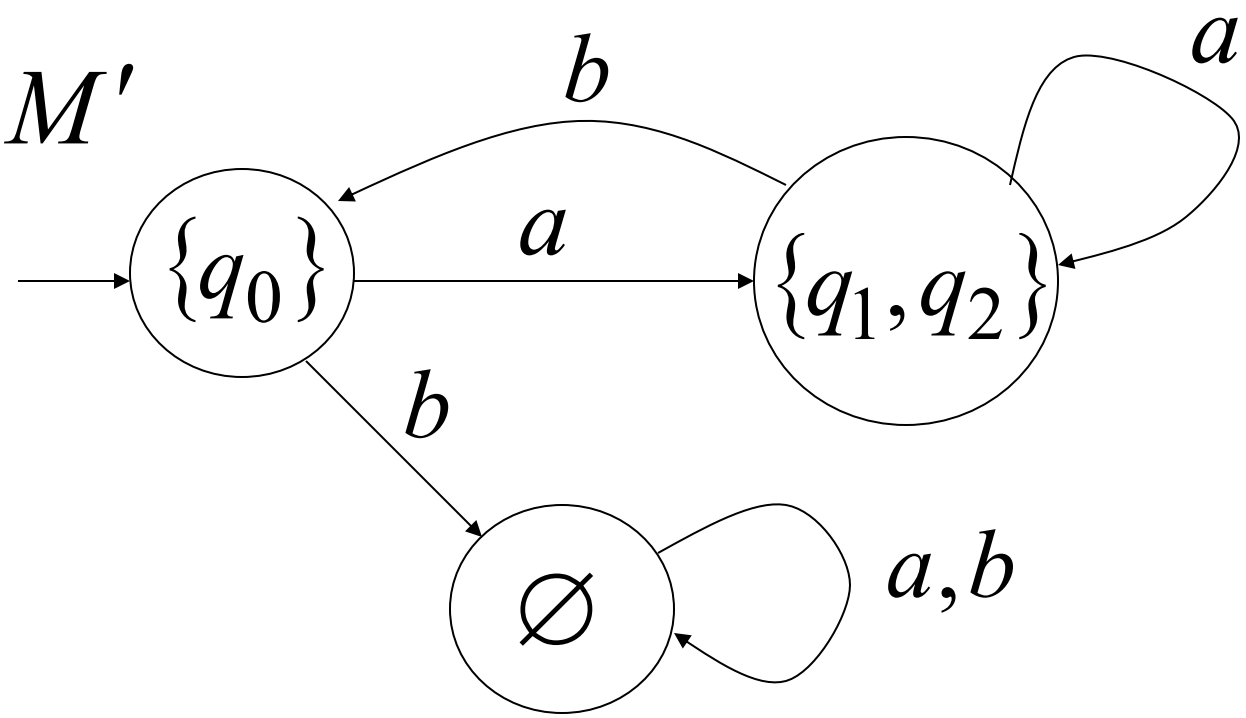
3. Repeat Step 2 for every state in DFA and symbols in alphabet until no more states can be added in the DFA

Example

NFA M



DFA M'



step

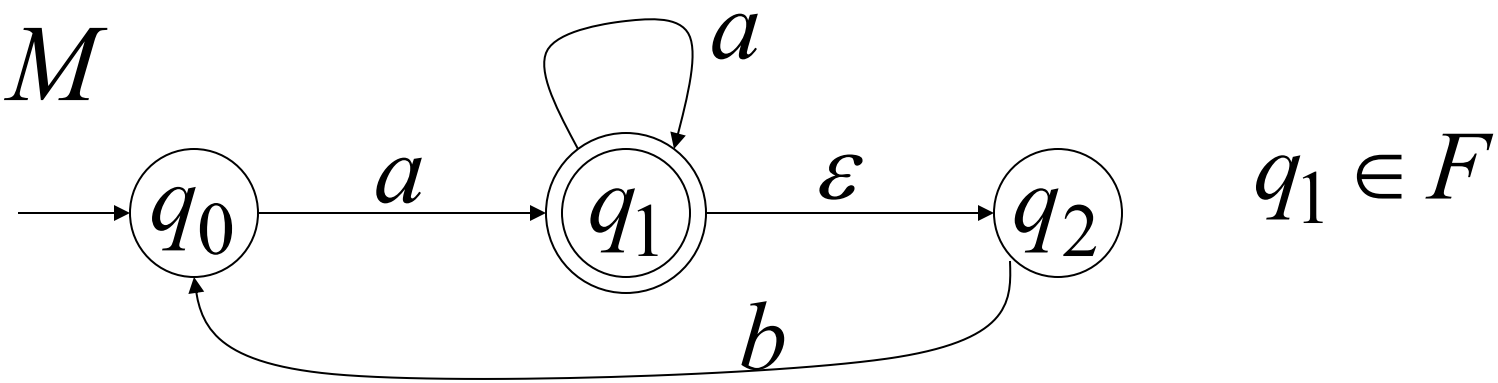
4. For any DFA state $\{q_i, q_j, \ldots, q_m\}$

if some q_j is accepting state in NFA

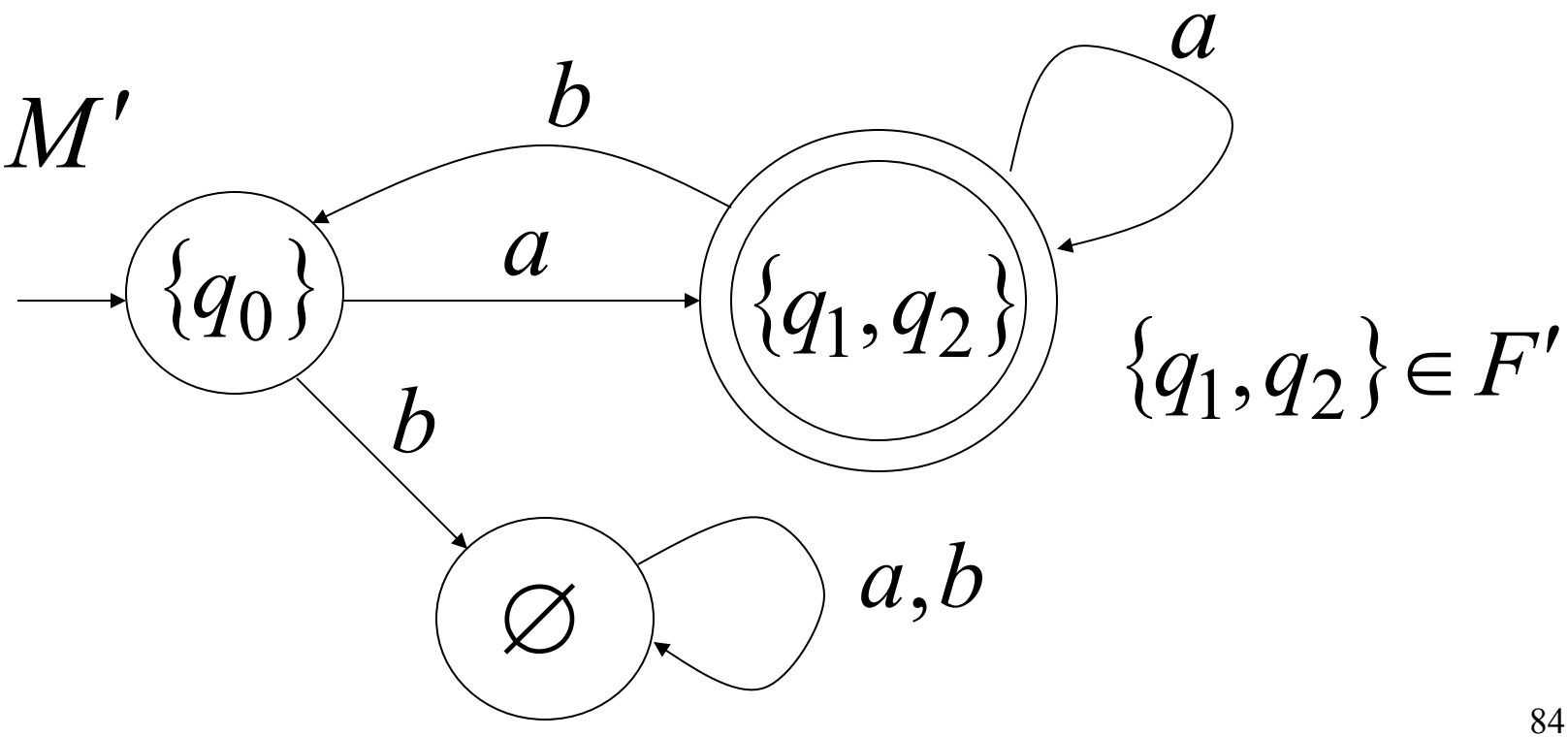
Then, $\{q_i, q_j, \ldots, q_m\}$
is accepting state in DFA

Example

NFA M



DFA M'



Corollary:

A language is regular if and only if some nondeterministic finite automaton recognizes it.

Proof:

- (If) A language is regular if some NFA recognizes it.
  - It was shown that any NFA can be converted into an equivalent DFA. Consequently, if an NFA recognizes some language, so does some DFA, and hence the language is regular.

Corollary:

A language is regular if and only if some nondeterministic finite automaton recognizes it.

Proof (Cont.):

- (Only If) a language is regular only if some NFA recognizes it.
  - A regular language has a DFA recognizing it and any DFA is also an NFA.